

2. Bölüm Ders Notu Derlemesi

Bölüm 2 – Uygulama Katmanı

Bu başlık slaytı, ağ mimarisine artık kullanıcıya en yakın yerden baktığını gösterir. Uygulama katmanı, ağın teknik olarak en üst katmanıdır; fakat öğrenme açısından yalnızca “üstte duran” bir katman değildir. Tam tersine, ağın neden var olduğunu açıklayan katmandır. Web, e-posta, video akışı, mesajlaşma, DNS çözümleme ve gerçek zamanlı iletişim gibi tüm görünür servisler burada anlam kazanır. Kitabın da vurguladığı gibi, ağ uygulamaları bilgisayar ağlarının varlık nedenidir; ağ altyapısı ve protokoller, sonunda bu uygulamaları taşıyabilmek için vardır.

Bu başlığı gördüğünde şunu zihninde netleştirmelisin: artık alt katmanların “nasıl ilettiği”nden çok, bu iletim üzerinde çalışan uygulamaların “nasıl tasarlandığı” ve “hangi hizmetlere ihtiyaç duyduğu” ile ilgileniyorsun. Yani odak, kablo veya yönlendirme değil; tarayıcı, sunucu, istemci, süreç (process), socket ve uygulama katmanı protokolüdür. Bu bölümün önemi de tam buradadır: soyut görünen ağ kavramlarını, doğrudan kullandığın servislerle ilişkilendirir.

Kısa Özet

Uygulama katmanı, ağın kullanıcıya görünen yüzüdür. Bu bölümde yalnızca protokol isimlerini değil, uygulamaların nasıl çalıştığını, hangi mimariyi kullandığını ve taşıma katmanından ne beklediğini anlamak gerekir.

Uygulama Katmanı: Genel Bakış

Bu slayt, bölümün kapsamını verirken aynı zamanda büyük resmi kurar. Uygulama mimarileri, HTTP, e-posta, DNS, P2P uygulamalar, video akışı, CDN ve socket programlama başlıkları aslında birbirinden kopuk konular değildir; hepsi aynı sorunun farklı cevaplarıdır: “Ağ üzerinde çalışan bir uygulama nasıl tasarlanır ve nasıl işler?”

Burada dikkat edilmesi gereken nokta, uygulama katmanının sadece “uygulama isimleri listesi” olmadığıdır. Örneğin HTTP yalnızca bir web protokolü değildir; istemci-sunucu modelini, mesaj biçimini, durumsuzluk (statelessness) ilkesini ve taşıma katmanı seçimini birlikte düşündürür. DNS yalnızca isim çözümleme sistemi değildir; dağıtık yapı, gecikme, önbellekleme ve ölçeklenebilirlik tartışmasını açar. Video akışı ve CDN konusu ise bant genişliği, gecikme, önbellekleme ve içerik dağıtım stratejilerini birlikte görünür kılar.

Bu slaytı bir yol haritası gibi düşünmelisin. Bölümün ilerleyen kısımlarında tek tek işlenecek her konu, aslında ağ uygulamalarının üç temel boyutuna bağlanır: mimari yapı, protokol davranışı ve uygulamanın taşıma katmanından beklediği hizmet modeli.

Kısa Özet

Bu bölüm, ağ uygulamalarını yalnızca örnekler üzerinden değil, ortak ilkeleri üzerinden

inceleyecektir. Ana hedefin, “hangi uygulama hangi protokolü kullanır?” sorusundan çok, “neden o protokolü ve mimariyi seçer?” sorusunu cevaplayabilmek olmalıdır.

Uygulama Katmanı: Genel Bakış – Hedeflerimiz

Bu slayt, uygulama katmanı protokollerini iki eksenle ele alır: kavramsal boyut ve uygulama boyutu. Kavramsal boyut, istemci-sunucu (client-server), eşler arası (peer-to-peer), süreçler (processes), socket ve servis gereksinimleri gibi temel fikirleri kapsar. Uygulama boyutu ise HTTP, SMTP, IMAP, DNS, video yayın sistemleri ve socket API gibi somut örnekleri içerir.

Burada önemli olan, soyut ile somut arasında bağ kurmaktır. Örneğin “client-server paradigm” yalnızca teorik bir mimari değildir; HTTP’nin neden bu kadar yaygın çalışabildiğini açıklar. Aynı şekilde “transport-layer service model” konusu yalnızca TCP ve UDP farkı değildir; bir uygulamanın güvenilirlik, zamanlama, verim (throughput) ve güvenlik ihtiyaçlarını nasıl değerlendirdiğini gösterir. Socket API ise bu kavramların programlama tarafındaki karşılığıdır; yani teorisinin koda geçtiği noktadır.

Bu slayta bakarken kendine şu soruyu sormalısın: Bir uygulama geliştiricisi olarak ağdan ne beklerim? Çünkü bölümün geri kalanı, bu soruya verilen farklı cevaplardan oluşur.

Kısa Özet

Bu bölümün hedefi yalnızca protokolleri tanımak değil, uygulama geliştiricisinin bakış açısından ağ uygulamalarını anlamaktır. Temel kavram, uygulama gereksinimi ile protokol seçimi arasındaki ilişkiyi kurabilmektir.

Bazı Ağ Uygulamaları

Bu slayt, ağ uygulamalarının çeşitliliğini göstererek uygulama katmanının ne kadar geniş bir alanı kapsadığını hatırlatır. Sosyal ağlar, web, mesajlaşma, e-posta, çok kullanıcıli oyunlar, video akışı, P2P paylaşım, VoIP, video konferans, arama motorları ve uzaktan erişim gibi uygulamalar ilk bakışta çok farklı görünür. Ancak ağ açısından bakıldığında bunların hepsi aynı temel probleme çözüm üretir: farklı uç sistemlerde çalışan süreçlerin bir protokol üzerinden anlamlı veri alışverişi yapması.

Burada asıl mesele, uygulamaların “aynı ağ” üzerinde ama “farklı ihtiyaçlarla” çalıştığını fark etmektir. E-posta uygulaması güvenilirlik ister ama yüksek gecikmeye duyarlı değildir. Canlı video konferans ise düşük gecikme ister ama küçük veri kayıplarını tolere edebilir. Web gezintisi çoğu zaman güvenilirlik ister, oyunlar ise gecikmeye çok daha duyarlıdır. Dolayısıyla uygulama katmanını anlamak, kullanım senaryosunu anlamayı gerektirir.

Bu kavramı şöyle düşünebilirsin: Aynı otoyolu kullanan araçlar gibi, tüm uygulamalar aynı İnternet altyapısını kullanır; fakat kamyonla ambulansın beklentisi aynı değildir. Ağ uygulamaları da farklı servis beklentileriyle tasarlanır.

Kısa Özet

Ağ uygulamaları çok çeşitlidir; fakat hepsi süreçler arası iletişim üzerine kuruludur. Farkı

yaratın şey, her uygulamanın güvenilirlik, gecikme, verim ve güvenlik gereksinimlerinin farklı olmasıdır.

Ağ Uygulaması Nasıl Geliştirilir?

Bu slayt, ağ uygulaması geliřtirmenin mimari olarak nerede gerekleřtiđini açıklar. Temel fikir řudur: ağ uygulaması geliřtirirken yönlendiricilere, anahtarlara veya ağ ekirdeđine yazılım yazmazsın; yazılımı uç sistemlerde, yani end systems üzerinde geliřtirirsin. Tarayıcı, mobil uygulama, istemci programı, sunucu uygulaması veya ierik sunucusu gibi bileřenler hep uta alıřır. Ağ ekirdeđi ise bu ulardaki süreçlerin mesajlarını tařır. Kitap da özellikle bunu vurgular: uygulama yazılımının uç sistemlerle sınırlı olması, yeni uygulamaların hızla geliřtirilebilmesini ve yaygınlařabilmesini sađlamıřtır.

Burada önemli olan, application architecture ile network architecture kavramlarını karıřtırmamaktır. Network architecture, İnternet'in katmanlı yapısı gibi genel ve sabit altyapıyı anlatır. Application architecture ise uygulama geliřtiricisinin tasarladıđı yapıdır. Yani “istemci mi olacak, sunucu mu olacak, P2P mi olacak, hangi süreçler konuřacak?” soruları application architecture kapsamındadır.

řekilde farklı ağlar — mobil ağ, ev ağ, kurumsal ağ, ISP ve veri merkezi — gösterilirken vurgulanan şey, iletiřimin uygulama katmanında utan ua gerekleřmesidir. Aradaki ağlar uygulamayı bilmez; sadece tařıma ve yönlendirme iřini yapar.

Kısa Özet

Ağ uygulaması geliřtirmek, ağ ekirdeđine kod yazmak deđil, uç sistemlerde alıřan yazılımlar geliřtirmektir. Bu ayrım, İnternet üzerinde yeni uygulamaların hızlı biimde ortaya ıkabilmesinin temel nedenlerinden biridir.

İstemci-Sunucu Modeli

İstemci-sunucu (client-server) modeli, modern ağ uygulamalarının en temel mimari desenlerinden biridir. Bu modelde sürekli açık olan, genellikle sabit IP adresine sahip ve çođu zaman veri merkezinde alıřan bir sunucu vardır. İstemci ise bu sunucuya bađlanan, zaman zaman evrimdiři olabilen, dinamik IP kullanabilen ve hizmet talep eden taraftır. HTTP, IMAP ve FTP gibi klasik uygulamalar bu mimariyi kullanır. Kitap da istemci-sunucu mimarisinin temel özelliklerini “always-on server”, “well-known IP address” ve istemcilerin birbirleriyle dođrudan konuřmaması řeklinde ereveseler.

Bu mimarinin gücü merkezileřmeden gelir. Sunucu tarafında kontrol, veri yönetimi, kimlik dođrulama ve ierik sunumu daha kolaydır. Ancak bu merkezileřme aynı zamanda öleklenme baskısı yaratır. Tek bir sunucu ok sayıda istemci talebini karřılayamayabilir; bu nedenle pratikte veri merkezleri, yük dengeleme (load balancing) ve sunucu kümeleri devreye girer.

Burada şunu anlamalısın: istemci ve sunucu rolleri “daha zayıf” ve “daha güçlü” cihaz ayrımı değildir; iletişimi başlatan taraf ile hizmet bekleyen taraf ayrımıdır. Bir akıllı telefon istemci olabilir, ama çok güçlü bir bilgisayar da istemci rolünde olabilir.

Kısa Özet

İstemci-sunucu modeli, merkezi ve yönetilebilir bir mimari sunar. İstemci talebi başlatır, sunucu hizmet verir; bu yapı basit ve yaygındır, ancak yüksek ölçeklerde güçlü altyapı gerektirir.

P2P (Peer-to-Peer) Modeli

P2P modelini anlamamanın en iyi yolu, onu istemci-sunucu modelinin simetriği olarak düşünmektir. Burada sürekli açık merkezi bir sunucu zorunlu değildir; uç sistemler doğrudan birbirleriyle konuşur ve aynı düğüm hem hizmet alabilir hem de hizmet verebilir. BitTorrent bunun klasik örneğidir. Kitap, P2P mimarisinin en dikkat çekici avantajını self-scalability olarak açıklar: sisteme yeni katılan her eş (peer), yalnızca yük getirmez; aynı zamanda kapasite de ekler.

Bu model özellikle içerik dağıtımı gibi senaryolarda etkileyicidir. Merkezî bir sunucunun her şeyi dağıtması yerine, dosyayı alan eşler dosyayı diğer eşlere de aktarır. Böylece toplam sistem kapasitesi artar. Fakat bu esnekliğin bedeli vardır: güvenlik, yönetim, performans öngörülebilirliği ve güvenilirlik daha karmaşık hale gelir.

Burada karıştırılabilecek nokta şudur: P2P tamamen “sunucusuz” anlamına gelmek zorunda değildir. Bazı P2P sistemlerinde eşlerin birbirini bulması için kısmi merkezi bileşenler olabilir. Asıl ayırt edici özellik, veri alışverişinin büyük ölçüde eşler arasında doğrudan yapılmasıdır.

Kısa Özet

P2P mimarisi, merkezi sunucuya olan bağımlılığı azaltır ve ölçeklenebilirliği artırır. Ancak bu esneklik, güvenlik ve yönetim açısından daha karmaşık bir yapı doğurur.

Süreçler (Processes) Arası İletişim

Ağ uygulamalarında aslında konuşan şey “bilgisayarlar” değil, o bilgisayarlarda çalışan süreçlerdir (processes). Process, çalışan programdır. Aynı bilgisayardaki süreçler işletim sistemi tarafından sağlanan mekanizmalarla haberleşebilir; fakat ağ bağlamında asıl önemli olan, farklı host’larda çalışan süreçlerin mesaj alışverişiyle iletişim kurmasıdır. Kitap bu ayrımı özellikle vurgular: ağ uygulaması, süreç çiftleri arasındaki mesajlaşma üzerine kurulur.

Burada client process ve server process tanımları önemlidir. İletişimi başlatan süreç client’tır; bağlantı bekleyen süreç server’dır. Bu tanım rol bazlıdır, cihaz bazlı değil. Bir P2P sisteminde aynı eş, bir oturumda client, başka bir oturumda server olabilir. Bu yüzden “client” ve “server” kavramlarını fiziksel makineyle değil, iletişim oturumu içindeki rolle ilişkilendirmen gerekir.

Bu kavramı doğru anlarsan, sonraki socket ve adresleme konuları çok daha netleşir. Çünkü aslında ağ, bir sürecin başka bir sürece nasıl ulaşacağını çözmeye çalışır.

Kısa Özet

Ağ uygulamalarında iletişim, süreçler arasında gerçekleşir. Client, iletişimi başlatan süreçtir; server ise iletişim bekleyen süreçtir. Bu roller oturum bağlamında tanımlanır.

Socket Kavramı

Socket, uygulama ile ağ arasındaki yazılımsal kapıdır. Bir süreç mesaj göndermek istediğinde bunu doğrudan ağa “itmez”; önce socket’e yazar. Karşı tarafta da mesaj, alıcı sürecin socket’i üzerinden teslim alınır. Kitap bu ilişkiyi çok güçlü bir benzetmeyle açıklar: süreç bir ev ise, socket o evin kapısıdır. Mesaj kapıdan çıkar, ağ üzerinden taşınır ve hedefte başka bir kapıdan içeri girer.

Burada kritik ayırım, kontrol sınırındır. Uygulama geliştiricisi, socket’in uygulama tarafını kontrol eder; ancak socket’in altındaki taşıma mekanizmasının büyük kısmı işletim sistemi ve seçilen protokol tarafından yönetilir. Sen uygulama olarak TCP mi UDP mi kullanacağını seçersin; fakat paketlerin ağda nasıl yönlendirileceğini belirlemezsin.

Şekilde “controlled by app developer” ve “controlled by OS” ayırımı bu yüzden önemlidir. Socket, yalnızca teknik bir arayüz değil; uygulama geliştiricisinin ağla temas ettiği temel programlama soyutlamasıdır. Daha sonra socket programming konusu geldiğinde, tüm pratik kurgu bu kavram üzerine oturacaktır.

Kısa Özet

Socket, uygulama ile taşıma katmanı arasındaki arayüzdür. Süreçler mesajlarını socket üzerinden gönderir ve alır; bu nedenle socket, ağ programlamasının temel yapı taşıdır.

Süreç Adresleme

Bir sürece mesaj gönderebilmek için yalnızca host’u bilmek yetmez; host içindeki doğru süreci de tanımlamak gerekir. Bu yüzden süreç adresleme, iki bileşenden oluşur: IP adresi ve port numarası. IP adresi hedef makineyi, port numarası ise o makinedeki hedef süreci veya daha doğru ifadeyle hedef socket’i belirtir. Örneğin bir web sunucusuna HTTP mesajı göndermek için IP adresi yanında port 80 bilgisi gerekir. Kitap da bunu açıkça vurgular: receiving process’i tanımlamak için host address + destination port number birlikte gereklidir.

Burada dikkat edilmesi gereken yanılğı şudur: “IP adresi zaten bilgisayarı gösteriyor; neden port gerekiyor?” Çünkü aynı bilgisayarda aynı anda birden fazla ağ uygulaması çalışabilir. Web sunucusu, posta sunucusu, DNS hizmeti veya başka uygulamalar aynı host üzerinde olabilir. Ağın yapması gereken şey yalnızca “doğru binaya” değil, “doğru daireye” teslim yapmaktır. IP adresi bina, port numarası daire gibi düşünülebilir.

Bu kavram sonraki tüm istemci-sunucu iletişiminin temelidir. URL, DNS çözümleme ve HTTP istekleri gibi pek çok şey sonunda IP + port eşleşmesine dayanır.

Kısa Özet

Bir sürece ulaşmak için yalnızca IP adresi yeterli değildir; port numarası da gerekir. IP host'u, port ise host içindeki hedef uygulama sürecini tanımlar.

Uygulama Katmanı Protokolü Neleri Tanımlar? – Mesaj Türleri ve Sözdizimi

Uygulama katmanı protokolü, iki sürecin yalnızca “iletişim kuracağını” söylemez; bu iletişimin nasıl yapılacağını ayrıntılı biçimde tanımlar. Öncelikle hangi mesaj türlerinin değiş tokuş edileceği belirlenir: istek (request) ve yanıt (response) gibi. Ardından bu mesajların sözdizimi (syntax) tanımlanır; yani mesajın hangi alanlardan oluştuğu, bu alanların sırası, sınırları ve biçimi belirlenir. HTTP örneğinde GET isteği, request line ve header satırları bunun tipik göstergesidir.

Syntax kavramını “mesajın grameri” gibi düşünebilirsin. İnsan dilinde kelimeler rastgele sıralanmaz; burada da protokol mesajları rastgele kurulmaz. Örneğin `GET /index.html HTTP/1.1` satırı yalnızca bilgi taşımaz; aynı zamanda standart bir yapı izler. Eğer istemci ve sunucu bu yapıda anlaşılmazsa, iletişim gerçekleşse bile anlamlı olmaz.

Burada önemli olan, protokolün “veri taşımak” ile “anamlı iletişim kurmak” arasındaki farkı kapatmasıdır. Taşıma katmanı baytları iletir; uygulama katmanı protokolü ise o baytların ne anlama geldiğini yapılandırır.

Kısa Özet

Uygulama katmanı protokolü, hangi mesajların gönderileceğini ve bu mesajların nasıl biçimleneceğini tanımlar. Syntax, mesajın grameridir; anlamlı ve birlikte çalışabilir iletişim bunun üzerine kurulur.

Uygulama Katmanı Protokolü Neleri Tanımlar? – Semantik ve Mesajlaşma Kuralları

Bir protokolde yalnızca biçim yetmez; anlam da gerekir. Mesaj semantiği, bir alanın ne ifade ettiğini tanımlar. Örneğin HTTP’de `200 OK`, isteğin başarıyla işlendiğini; `404 Not Found` ise istenen kaynağın bulunmadığını bildirir. Yani syntax “mesaj nasıl yazılır?” sorusunu, semantik ise “mesaj ne anlama gelir?” sorusunu cevaplar.

Bunun yanında protokol, mesajlaşma kurallarını da tanımlar: mesaj ne zaman gönderilir, hangi mesaja hangi yanıt verilir, iletişim sırası nasıldır, bağlantı ne zaman kapanır? Bu kısım, protokolün davranış mantığıdır. Örneğin istemci bağlanır, istek gönderir, sunucu yanıt verir ve gerekirse bağlantı kapatılır. Bu akış kuralları olmazsa, iki taraf aynı dili konuşsa bile ne zaman konuşacaklarını bilemez.

Burada şunu net görmelisin: protokol, yalnızca paket biçimi değil; aynı zamanda davranış sözleşmesidir. Bu nedenle bir uygulama katmanı protokolünü anlamak, hem mesaj formatını hem de etkileşim sırasını anlamayı gerektirir.

Kısa Özet

Semantik, mesaj alanlarının ne anlama geldiğini açıklar; mesajlaşma kuralları ise iletişimin

hangi sırayla ilerleyeceğini belirler. Protokol, bu iki boyut birlikte tanımlandığında gerçekten çalışır.

Uygulama Katmanı Protokolü Neleri Tanımlar? – Açık ve Özel Protokoller

Bu slayt, protokol tasarımında standardizasyonun neden önemli olduğunu gösterir. Açık protokoller (open protocols), RFC gibi kamuya açık belgelerde tanımlanır; herkes bu protokolleri okuyabilir, uygulayabilir ve sistemler arasında birlikte çalışabilirlik (interoperability) sağlanır. HTTP, FTP ve SMTP bunun örnekleridir. Buna karşılık özel (proprietary) protokoller şirketlere aittir; ayrıntıları kamuya açık olmayabilir. Skype ve Zoom gibi sistemler buna örnek olarak verilir.

Burada asıl kavranması gereken nokta, açık protokolün teknik olduğu kadar ekosistem açısından da güçlü olduğudur. Açık standart sayesinde Chrome ve Firefox aynı web sunucusuna bağlanabilir. Farklı işletim sistemleri ve üreticiler aynı ağ üzerinde birlikte çalışabilir. Açık standartlar İnternet'in büyümesini hızlandıran temel unsurlardan biridir.

Özel protokoller ise belirli bir şirket için optimizasyon, kontrol ve farklılaştırma avantajı sağlayabilir; ancak birlikte çalışabilirlik ve şeffaflık bakımından daha sınırlı olabilir. Bu ayrım, yalnızca mühendislik değil, aynı zamanda strateji meselesidir.

Kısa Özet

Açık protokoller, birlikte çalışabilirliği ve yaygın benimsenmeyi destekler. Özel protokoller daha kapalıdır; belirli avantajlar sağlayabilirler ama ekosistem açıklığı açısından daha sınırlıdır.

Bir Uygulama Hangi Taşıma Servisine İhtiyaç Duyar? – Veri Bütünlüğü ve Zamanlama

Bu slayt, uygulama gereksinimi ile taşıma protokolü seçimi arasındaki temel ilişkiyi kurar. Her uygulama aynı tür taşıma hizmetine ihtiyaç duymaz. Bazı uygulamalar veri bütünlüğünü vazgeçilmez görür; dosya transferi, web işlemleri veya e-posta gibi uygulamalarda veri kaybı veya bozulma kabul edilemez. Buna karşılık VoIP veya gerçek zamanlı medya gibi uygulamalar küçük veri kayıplarını tolere edebilir; burada asıl kritik unsur düşük gecikmedir. Kitap da transport services boyutlarını reliable data transfer, timing, throughput ve security olarak sınıflandırır.

Burada dikkat edilmesi gereken nokta, “her zaman güvenilirlik en iyidir” düşüncesinin doğru olmamasıdır. Bir PDF dosyasında tek bit hatası bile önemli olabilir; ama canlı seste tek paket kaybı çoğu zaman fark edilmez. Buna karşılık 500 ms gecikme bir çevrim içi oyunu kullanılmaz hale getirebilir. Dolayısıyla asıl soru şudur: uygulama için hangisi daha kritik, doğruluk mu zamanlama mı?

Bu ayrım, uygulama katmanı tasarımının en önemli sezgilerinden biridir. Çünkü TCP ile UDP arasındaki tercih çoğu zaman bu ihtiyaç analizinden doğar.

Kısa Özet

Bazı uygulamalar için güvenilirlik, bazıları için gecikme daha kritiktir. Uygulama gereksinimini doğru okumadan taşıma protokolü seçimi sağlıklı yapılamaz.

Bir Uygulama Hangi Taşıma Servisine İhtiyaç Duyar? – Verim ve Güvenlik

Verim (throughput), belirli bir iletişim oturumunda göndericinin alıcıya bit iletebildiği hızdır. Bazı uygulamalar minimum verim ister; örneğin bir video akışının kabul edilebilir kaliteyle çalışması için belli bir Mbps düzeyine ihtiyacı olabilir. Bu tür uygulamalar bandwidth-sensitive olarak düşünülür. Buna karşılık e-posta veya dosya indirme gibi elastic applications, mevcut throughput neyse onu kullanır; daha fazlası daha iyidir ama alt eşik her zaman kritik değildir. Kitap bu ayrımı açık biçimde yapar.

Güvenlik tarafında ise şifreleme, veri bütünlüğü ve kimlik doğrulama öne çıkar. Burada kritik nokta şudur: güvenilirlik ile güvenlik aynı şey değildir. Bir veri kayıpsız ve sıralı ulaşabilir ama üçüncü kişiler tarafından okunuyor olabilir. Bu yüzden HTTP + TLS birleştğinde HTTPS ortaya çıkar. Güvenlik, çoğu zaman uygulama ve taşıma katmanı arasında ek bir mekanizma olarak düşünülmelidir.

Bu slayt, taşıma hizmetini yalnızca “TCP mi UDP mi?” sorusuna indirgememen gerektiğini gösterir. Aslında düşünmen gereken daha geniş bir çerçevedir: güvenilirlik, gecikme, verim ve güvenlik birlikte değerlendirilmelidir.

Kısa Özet

Verim, uygulamanın ne kadar veri hızına ihtiyaç duyduğunu; güvenlik ise verinin gizli, bütün ve doğrulanabilir kalmasını ilgilendirir. Uygulama tasarımında bu gereksinimler birlikte ele alınmalıdır.

Taşıma Hizmeti Gereksinimleri: Yaygın Uygulamalar

Bu tablo, teorik taşıma servisi boyutlarını gerçek uygulamalarla eşleştirir. Dosya transferi, e-posta ve web belgeleri için veri kaybı kabul edilemez; bunlar elastic uygulamalardır ve zaman hassasiyetleri düşük sayılır. Gerçek zamanlı ses/video ve oyunlar ise veri kaybına daha toleranslıdır ama gecikmeye duyarlıdır. Akıllı telefon mesajlaşması ise ilginç bir ara örnektir: veri kaybına toleransı yoktur; zaman hassasiyeti ise kullanım senaryosuna göre hem var hem yoktur. Kitaptaki Figure 2.4 de benzer şekilde seçilmiş uygulamaların loss, throughput ve time-sensitive gereksinimlerini karşılaştırır.

Tabloyu ezberlenecek bir liste gibi değil, karar matrisi gibi görmelisin. Bu tablo şunu öğretir: “uygulama türü → servis gereksinimi → protokol tercihi” şeklinde bir zincir vardır. Örneğin video konferans için düşük gecikme kritik olduğu için kayıp toleransı kabul edilir. Web için ise doğruluk ön plandadır.

Bu tablo, ileride neden bazı uygulamaların TCP, bazılarının UDP veya TCP+TLS gibi çözümler kullandığını anlamanın temelini kurar.

Kısa Özet

Yaygın ağ uygulamaları farklı taşıma ihtiyaçlarına sahiptir. Bu tablo, protokol seçiminde asıl belirleyicinin uygulamanın kullanım amacı ve kalite beklentisi olduğunu gösterir.

İnternet Taşıma Protokolleri Servisleri – TCP

TCP, uygulamalara connection-oriented ve reliable data transfer hizmeti sunar. Connection-oriented olması, veri akışı başlamadan önce istemci ile sunucu arasında bir handshaking yapılması anlamına gelir. Reliable data transfer ise verinin kaybolmadan, bozulmadan ve doğru sırada teslim edilmesini sağlar. Ek olarak TCP, flow control ile alıcı tamponunun aşılmasını önler ve congestion control ile ağda tıkanıklık oluştuğunda gönderim hızını azaltır. Kitap bu servis modelini özellikle vurgular.

Burada önemli olan, TCP'nin yalnızca “güvenilir” değil, aynı zamanda “byte-stream oriented” olduğunu da fark etmektir. Yani uygulamaya mesaj sınırlarıyla değil, düzenli bir bayt akışı gibi davranır. Bu ayrımı, uygulama protokolü tasarlarlarken önemlidir; çünkü mesaj sınırlarını uygulama katmanında sen yönetmek zorunda kalabilirsin.

TCP'nin sunduğu güvenilirlik, sequence number, ACK, retransmission ve timeout gibi mekanizmalarla sağlanır. Ancak bunların ayrıntısı daha sonra taşıma katmanında işlenecektir. Bu slaytta odak, TCP'nin uygulamaya hangi hizmet modelini sunduğunu kavramaktır.

Kısa Özet

TCP, bağlantı kurarak çalışan, güvenilir, sıralı ve akış/tıkanıklık kontrolü yapan bir taşıma protokolüdür. Uygulama düzeyinde en güçlü tarafı, güvenilir veri teslimini standart olarak sunmasıdır.

İnternet Taşıma Protokolleri Servisleri – TCP'nin Sağlamadıkları

Bu slayt çok değerlidir çünkü TCP hakkında yaygın yanlış anlamaları düzeltir. TCP güvenilirlik sağlar; fakat gecikme garantisi vermez, minimum bant genişliği garantisi vermez ve yerleşik şifreleme sağlamaz. Yani TCP “her şeyi çözen” bir protokol değildir. Kitap da TCP ve UDP'nin timing veya throughput guarantee sunmadığını açıkça söyler; günümüz İnternet'i zaman duyarlı uygulamalara çoğu zaman yeterli hizmet verse de bunu garanti etmez.

Burada en sık karıştırılan konu güvenilirlik ile güvenlidir. TCP veriyi kayıpsız ulaştırabilir ama açık metin olarak taşıyabilir. Aynı şekilde veri sıralı gelebilir ama çok geç gelebilir. Dolayısıyla “TCP kullanıyorum, sorun yok” demek eksik bir değerlendirmedir.

Bu kavramı doğru oturtursan, neden HTTPS için TLS gerektiğini, neden video uygulamalarının bazen TCP'den memnun olmadığını ve neden taşıma protokolü seçiminin uygulama hedeflerine bağlı olduğunu daha net görürsün.

Kısa Özet

TCP güçlü bir taşıma hizmeti sunar ama zamanlama, bant genişliği ve güvenlik garantisi vermez. Güvenilirlik ile güvenliği, teslim ile gecikmeyi birbirinden ayırman gerekir.

İnternet Taşıma Protokolleri Servisleri – UDP

UDP, TCP'ye göre çok daha hafif, connectionless ve unreliable bir taşıma protokolüdür. Önceden bağlantı kurulmaz; süreç mesajını UDP socket'e verir ve UDP bunu ağ katmanına iletir. Ancak mesajın hedefe ulaşacağını, sıralı geleceğini veya yeniden iletim yapılacağını garanti etmez. Kitap UDP'yi “no-frills, lightweight transport protocol” olarak tanımlar.

Burada önemli olan, UDP'nin eksik bir TCP gibi düşünülmemesidir. UDP, bazı uygulamalar için bilinçli bir tercihtir. Daha düşük protokol yükü (header overhead), bağlantı kurulumu olmaması ve uygulama katmanında özel kontrol mantıkları kurma esnekliği sağlar. Bu yüzden VoIP, oyunlar, canlı yayın ve bazı modern uygulama tasarımları için anlamlı olabilir.

Şunu net görmelisin: UDP'nin “güvenilir olmaması”, onun kötü olduğu anlamına gelmez; sadece uygulamaya daha fazla sorumluluk bıraktığı anlamına gelir. Uygulama gecikmeyi güvenilirliğe tercih ediyorsa UDP avantajlı hale gelebilir.

Kısa Özet

UDP bağlantısız, hafif ve güvenilirlik garantisi vermeyen bir taşıma protokolüdür. Gücü, sadeliğinden ve uygulamaya özel kontrol mekanizmaları kurmaya izin vermesinden gelir.

İnternet Uygulamaları ve Taşıma Protokolleri

Bu slayt, uygulama katmanı protokolü ile taşıma katmanı protokolü arasındaki ilişkiyi somutlaştırır. FTP, SMTP ve HTTP gibi uygulamalar TCP kullanır; çünkü veri kaybını tolere etmezler. İnternet telefonisi veya oyunlar gibi uygulamalar ise senaryoya göre UDP ya da TCP kullanabilir. Bu eşleştirme, teorik bilgilerin pratik karşılığıdır. Kitap da bazı popüler uygulamaların TCP veya UDP kullanım tercihini bu gereksinimler üzerinden açıklar.

Buradaki asıl fikir, “uygulama protokolü” ile “taşıma protokolü”nün aynı şey olmadığıdır. HTTP bir application-layer protocol'dür; TCP ise transport protocol'dür. SMTP uygulama protokolüdür; TCP onun altında çalışan taşıma çözümüdür. Bu katman ayrımını doğru kurman gerekir.

Ayrıca bu tablo, tek bir doğru seçim olmadığını da gösterir. Aynı uygulama ailesi farklı koşullarda farklı taşıma seçimleri yapabilir. Çünkü tasarım kararı her zaman kalite beklentisi, kayıp toleransı, gecikme hassasiyeti ve güvenlik gereksinimiyle birlikte düşünülür.

Kısa Özet

Uygulama protokolü ile taşıma protokolü farklı katmanlara aittir ama birbirine sıkı bağlıdır. Uygulamanın gereksinimi, altta TCP mi UDP mi kullanılacağını büyük ölçüde belirler.

TCP'nin Güvenli Hale Getirilmesi (Securing TCP)

Bu slayt, güvenilirlik ile güvenlik arasındaki farkı çok net çizdiği için kritiktir. Standart TCP ve UDP socket'leri şifreleme sağlamaz; veriler ağ üzerinde cleartext olarak taşınabilir. Bu nedenle parola gibi hassas bilgiler aradaki bağlantılarda okunabilir. Kitap da bunu doğrudan söyler: vanilla TCP and UDP sockets provide no encryption. TLS ise TCP üzerine eklenen bir güvenlik katmanı olarak encryption, integrity ve end-point authentication sağlar. Ancak kitap burada önemli bir ayrım yapar: TLS, TCP ile aynı seviyede üçüncü bir taşıma protokolü değildir; uygulama katmanında TCP'nin üzerine eklenen bir geliştirmedir.

Burada dikkat edilmesi gereken nokta, "TCP güvenilir → o halde güvenlidir" düşüncesinin yanlış olmasıdır. Güvenilirlik, verinin kaybolmadan ve sıralı gelmesiyle ilgilidir. Güvenlik ise verinin okunamaması, değiştirilememesi ve karşı tarafın kimliğinin doğrulanabilmesiyle ilgilidir. Bunlar farklı problemlerdir.

Bu yüzden HTTPS'i yalnızca "HTTP'nin güvenli versiyonu" diye ezberlemek yerine, "HTTP'nin TCP üzerinde, TLS ile korunmuş hali" olarak anlamak daha doğrudur.

Kısa Özet

TCP güvenilirlik sağlar ama yerleşik güvenlik sağlamaz. TLS, TCP üzerine eklenerek şifreleme, bütünlük ve kimlik doğrulama hizmetleri sunar; güvenilirlik ile güvenliği birbirinden ayırman gerekir.

Uygulama Katmanı: Genel Bakış – Web ve HTTP'ye Geçiş

Bu yol haritası slaytı, artık genel ilkelerden özel bir uygulamaya, yani Web ve HTTP dünyasına geçtiğini gösterir. Bu geçiş önemlidir; çünkü web, istemci-sunucu mimarisini, URL kavramını, stateless protokol fikrini, TCP kullanımını ve uygulama katmanı mesajlarını aynı anda görmeni sağlar. Başka bir deyişle, şimdiye kadar konuştuğun kavramların hepsi HTTP örneğinde somutlaşacaktır.

Burada yapman gereken şey, yeni konuya "sıfırdan" başlamamak. İstemci-sunucu modeli zaten tanımlandı, süreç ve socket kavramı işlendi, taşıma servisi gereksinimleri konuşuldu. Şimdi bunların hepsi Web üzerinde birleşecek.

Kısa Özet

Bu slayt, genel uygulama katmanı ilkelerinden HTTP örneğine geçişi işaret eder. Bundan sonra kavramları somut bir uygulama üzerinden daha net göreceksin.

Web ve HTTP

Web sayfasını tek bir dosya gibi düşünmek yanıltıcı olabilir. Aslında bir web sayfası, temel HTML dosyası ve ona referans veren birçok nesneden (object) oluşur: JPEG görüntüler, ses dosyaları, komut dosyaları, videolar ve benzeri içerikler. Her nesne bir URL ile adreslenebilir. Slayttaki URL örneğinde hostname ve pathname ayrımı özellikle gösterilir; bu, kaynağın hangi host üzerinde ve hangi yol altında bulunduğunu anlamayı sağlar.

Burada dikkat edilmesi gereken nokta, “web sayfası” ile “web nesnesi” arasındaki farktır. Tarayıcı çoğu zaman önce temel HTML’yi alır, sonra HTML içindeki referanslara bakarak başka nesneleri ister. Bu nedenle web erişimi tek bir dosya alışverişinden ibaret değildir; çoğu zaman çok nesneli bir uygulama davranışıdır.

Bu ayrım ileride non-persistent ve persistent HTTP bağlantılarını anlamak için çok önemlidir. Çünkü bağlantı maliyetini aslında nesne sayısı belirgin biçimde etkiler.

Kısa Özet

Web sayfası, çoğu zaman tek bir nesne değil, bir HTML dosyası ve onun referans verdiği birçok nesnenin toplamıdır. HTTP’nin çalışma biçimini anlamak için “sayfa” ile “nesne” ayrımını net kurmalısın.

HTTP Genel Bakış

HTTP, Web’in application-layer protocol’üdür. İstemci-sunucu modeline dayanır: istemci genellikle tarayıcıdır, sunucu ise istenen web nesnelerini barındıran web sunucusudur. Tarayıcı istek (HTTP request) gönderir, sunucu yanıt (HTTP response) döner. Bu kadar basit görünen akış aslında web’in temel çalışma modelidir.

Burada dikkat edilmesi gereken nokta, HTTP’nin “içeriği” değil “erişim protokolünü” tanımladığıdır. Tarayıcının nasıl render yaptığı, sayfanın nasıl görüldüğü veya JavaScript’in ne yaptığı HTTP’nin kendisi değildir. HTTP, nesnelerin nasıl isteneceğini ve nasıl teslim edileceğini belirler.

Şekilde farklı istemciler — örneğin Safari ve Firefox — aynı sunucuya istek gönderir. Bu da açık protokollerin birlikte çalışabilirlik avantajını görünür kılar. Farklı istemciler aynı HTTP kurallarını kullanarak aynı sunucuyla konuşabilir.

Kısa Özet

HTTP, Web üzerindeki istemci-sunucu iletişimini tanımlayan uygulama katmanı protokolüdür. Tarayıcı ister, sunucu yanıt verir; web’in temel etkileşim modeli bu akış üzerine kuruludur.

HTTP Genel Bakış (devam)

Bu slayt, HTTP’nin TCP kullandığını ve stateless olduğunu vurgular. HTTP istemcisi, sunucuya bir TCP bağlantısı başlatır; sunucu bu bağlantıyı kabul eder; ardından HTTP mesajları bu TCP bağlantısı üzerinden değiş tokuş edilir ve gerekirse bağlantı kapatılır. Buradaki kritik ifade “HTTP is stateless” cümlesidir. Yani sunucu, geçmiş istemci istekleri hakkında varsayılan olarak durum bilgisi tutmaz.

Statelessness kavramını doğru anlamalısın. Bu, sunucunun hiçbir zaman hiçbir bilgi tutamayacağı anlamına gelmez; yalnızca HTTP protokolünün kendisinin geçmiş isteklere dayalı durum tutmayı zorunlu kılmadığını anlatır. Bu sadelik, tasarımı kolaylaştırır ve ölçeklenebilirliği artırır; çünkü sunucu, her istemci için sürekli durum yönetmek zorunda

kalmaz. Buna karşılık oturum yönetimi gerektiğinde çerezler (cookies) gibi ek mekanizmalar devreye girer.

Bu slayt aynı zamanda şunu da ima eder: HTTP'nin mantığını anlamak için altındaki TCP bağlantısının varlığını unutmamalısın. HTTP mesajları kendi başına havada dolaşmaz; TCP üzerinde taşınır.

Kısa Özet

HTTP, mesajlarını TCP üzerinden taşır ve temel olarak stateless bir protokoldür. Bu sadelik, web'in ölçeklenebilirliğini artırır; fakat durum yönetimi gerektiğinde ek mekanizmalar gerekir.

HTTP Bağlantıları: İki Tür

HTTP'de bağlantılar iki ana biçimde ele alınır: kalıcı olmayan (non-persistent) ve kalıcı (persistent). Non-persistent HTTP'de her TCP bağlantısı üzerinden en fazla bir nesne taşınır; nesne geldikten sonra bağlantı kapanır. Eğer sayfada çok sayıda nesne varsa, her nesne için yeni bağlantı gerekir. Persistent HTTP'de ise tek TCP bağlantısı açık tutulur ve aynı istemci-sunucu arasında birden fazla nesne bu bağlantı üzerinden gönderilebilir.

Burada dikkat edilmesi gereken nokta, performans farkının yalnızca “daha az bağlantı” meselesi olmamasıdır. Her yeni TCP bağlantısı, hem RTT maliyeti doğurur hem de işletim sistemi ve sunucu tarafında ek yük oluşturur. Persistent bağlantı, bu maliyetleri azaltır. Özellikle çok nesneli sayfalarda bu fark belirgin hale gelir.

Bu konuyu şöyle düşünebilirsin: Her nesne için yeni kapı açıp kapatmak yerine, kapıyı bir süre açık tutup peş peşe teslimat yapmak daha verimlidir. HTTP 1.1'in kalıcı bağlantıları yaygınlaştırmasının temel nedeni budur.

Kısa Özet

Non-persistent HTTP her nesne için ayrı bağlantı açar; persistent HTTP ise aynı bağlantıyı birden fazla nesne için kullanır. Bu fark, gecikme ve sistem yükü üzerinde doğrudan etkilidir.

Kalıcı Olmayan HTTP: Örnek

Bu slayt, non-persistent HTTP'nin adım adım nasıl çalıştığını gösterir. Süreç şu mantıkla ilerler: istemci URL girer, TCP bağlantısını başlatır, sunucu bağlantıyı kabul eder, istemci HTTP request mesajını yollar, sunucu istenen nesneyi içeren response döner ve ardından bağlantı kapanır. Burada hem TCP kurulumu hem de HTTP mesaj alışverişi ayrı ayrı zaman ve kaynak maliyeti üretir.

Bu süreçte önemli olan, HTTP isteğinin yalnızca “dosyayı bana ver” demek değil, bir URL içindeki belirli kaynağı istemek olmasıdır. Sunucu gelen isteği yorumlar, uygun nesneyi bulur ve yanıt üretir. Eğer temel HTML dosyası başka nesnelere referans veriyorsa, bu yalnızca başlangıçtır.

Şekilde zaman eksenini kullanılması çok yerindedir; çünkü bu konu zaman maliyetini anlamadan kavranamaz. Non-persistent HTTP'nin temel sorunu, çok nesneli sayfalarda bu adımların tekrar tekrar yaşanmasıdır.

Kısa Özet

Non-persistent HTTP'de her nesne için TCP kurulumu ve HTTP alışverişi yeniden yapılır. Bu yapı basittir ama çok nesneli sayfalarda gecikme maliyetini artırır.

Kalıcı Olmayan HTTP: Örnek (devam)

Bu slayt, temel HTML geldikten sonra neler olduğunu açıklar. İstemci önce HTML dosyasını alır ve ayrıştırır; ardından HTML içinde referans verilen 10 JPEG nesnesini bulur. Sonra her nesne için benzer istek süreci yeniden başlar. Yani kullanıcı aslında “bir sayfa” istemiş olsa da ağ açısından bakıldığında çok sayıda nesne için çok sayıda istek yapılıyor olabilir.

Burada dikkat edilmesi gereken nokta, web performansının yalnızca dosya boyutuyla belirlenmediğidir. Nesne sayısı da belirleyicidir. Küçük ama çok sayıda nesne içeren bir sayfa, büyük ama az nesneli bir sayfadan daha farklı davranabilir; çünkü bağlantı ve RTT maliyetleri tekrar eder.

Bu slayt, tarayıcının işinin yalnızca gelen dosyayı göstermek olmadığını da hatırlatır. Tarayıcı aynı zamanda HTML'yi ayrıştırır, nesne referanslarını bulur ve bunlar için yeni istekler planlar.

Kısa Özet

Web sayfası çoğu zaman temel HTML'den fazlasıdır; HTML alındıktan sonra referans verilen diğer nesneler de ayrıca istenir. Bu nedenle sayfa yükleme maliyeti nesne sayısı ile yakından ilişkilidir.

Kalıcı Olmayan HTTP: Yanıt Süresi

Bu slayt, non-persistent HTTP'nin tepki süresini sezgisel değil, nicel olarak düşünmeni sağlar. RTT, küçük bir paketin istemciden sunucuya gidip geri dönmesi için gereken süredir. Bir nesne için toplam HTTP yanıt süresi, yaklaşık olarak bir RTT TCP bağlantısını başlatmak için, bir RTT de HTTP isteği ve yanıtın ilk baytlarının gelmesi için olmak üzere **2 RTT + dosya aktarım süresi** şeklinde yorumlanır. Slayttaki zaman diyagramı tam olarak bunu görselleştirir.

Bu formülü ezberlemekten çok yorumlamalısın. Dosya çok küçük olsa bile RTT büyükse toplam gecikme yüksek olabilir. Özellikle uzak sunucularla iletişimde veya çok sayıda küçük nesne indirildiğinde, dosya aktarım süresinden çok bağlantı ve istek-yanıt gecikmesi baskın hale gelir. Bu, web performansında neden RTT'nin bu kadar kritik olduğunu açıklar.

Burada önemli olan, formülün fiziksel bir yasa değil, gecikmenin bileşenlerini ayıran bir model olduğudur. Bu model, sonraki persistent HTTP avantajını daha iyi görmeyi sağlar.

Kısa Özet

Kalıcı olmayan HTTP’de her nesne için yaklaşık 2 RTT + dosya aktarım süresi kadar maliyet düşünülür. Bu da çok nesneli sayfalarda RTT etkisini büyütür ve toplam gecikmeyi artırır.

Kalıcı HTTP (HTTP 1.1)

Bu slayt, non-persistent HTTP’nin sorunlarını ve persistent HTTP’nin neden geliştirildiğini açıklar. Non-persistent yapı, nesne başına 2 RTT maliyeti getirir ve her bağlantı için ek işletim sistemi yükü oluşturur. Tarayıcılar bunu azaltmak için paralel TCP bağlantıları açsa da bu çözüm her zaman ideal değildir. Persistent HTTP ise bağlantıyı açık tutar; aynı istemci ile sunucu arasında birden fazla nesne aynı TCP bağlantısı üzerinden gönderilebilir. Bu yaklaşım, yanıt süresini ciddi biçimde azaltır.

Burada asıl kavram, maliyeti bağlantı başına değil, oturum başına düşünmektir. Eğer aynı sunucudan çok sayıda nesne alınacaksa, tek bağlantı üzerinden devam etmek çok daha mantıklıdır. Böylece handshake ve tekrar tekrar bağlantı açma maliyeti azalır.

Bu farkı sezgisel olarak şöyle düşünebilirsin: Aynı depodan peş peşe 20 paket alacaksan, her paket için yeniden giriş çıkış yapmak yerine bir kez içeri girip işi topluca bitirmek daha verimlidir. Persistent HTTP tam olarak bunu yapar.

Kısa Özet

Persistent HTTP, aynı TCP bağlantısını birden fazla nesne için kullanarak RTT ve bağlantı yükünü azaltır. Bu yüzden çok nesneli web sayfalarında non-persistent yapıya göre daha verimlidir.

HTTP İstek Mesajı

HTTP’de iki temel mesaj türü vardır: request ve response. Bu slayt request message yapısına odaklanır. HTTP istek mesajı ASCII tabanlı, yani insan tarafından okunabilir biçimde tasarlanmıştır. İlk satır request line’dır; burada method, path ve HTTP sürümü yer alır. Sonraki satırlar header alanlarıdır ve isteğe ilişkin ek bilgiler taşır. Örneğin Host, User-Agent gibi alanlar sunucunun isteği bağlama yerleştirmesine yardımcı olur.

Burada dikkat edilmesi gereken nokta, HTTP isteğinin salt veri değil, yapılandırılmış bir komut olmasıdır. GET /index.html HTTP/1.1 satırı, hem hangi kaynağın istendiğini hem de hangi protokol sürümünün kullanıldığını belirtir. Header alanları ise isteğin bağlamını tamamlar.

Bu slaytı anlamak, uygulama katmanı protokol tasarımının somut örneğini görmek açısından çok değerlidir. Çünkü burada daha önce konuştuğun syntax, semantics ve messaging rules kavramları gerçek bir protokol üzerinde görünür hale gelir.

Kısa Özet

HTTP istek mesajı, belirli bir sözdizimine sahip yapılandırılmış bir uygulama katmanı

mesajdır. Request line ve header alanları birlikte, istemcinin ne istediğini ve bu isteğin hangi bağlamda yapıldığını tanımlar.

HTTP İstek Mesajı: Genel Biçim

Bu slaytta artık HTTP’yi yalnızca “web protokolü” olarak değil, biçimsel bir uygulama katmanı protokolü olarak görmeye başlamalısın. HTTP istek mesajı rastgele bir metin değildir; belirli bir sözdizimine (syntax) göre kurulur. En üstte istek satırı (request line) vardır; burada method, URL ve protokol sürümü yer alır. Altında başlık satırları (header lines) gelir. Gerekirse en altta gövde (entity body) bulunur. Ayrıca satır sonlarının CRLF (\r\n) ile işaretlenmesi, protokolün metin tabanlı biçiminin önemli bir parçasıdır.

Burada dikkat edilmesi gereken nokta, isteğin üç farklı rolü aynı anda taşınmasıdır. Method, “hangi işlemi yapmak istediğini”; URL, “hangi kaynağı hedeflediğini”; version ise “hangi HTTP kural setiyle konuştuğunu” belirtir. Header alanları bağlam bilgisini ekler. Bu nedenle HTTP isteği yalnızca veri taşımaz; niyeti, hedefi ve bağlamı birlikte ifade eder. Bu, uygulama katmanı protokolü tasarımının çok tipik bir örneğidir.

Bir başka önemli ayrım da şudur: tüm HTTP istekleri gövde taşımak zorunda değildir. Örneğin klasik GET isteklerinde çoğu zaman asıl bilgi request line ve header alanlarında taşınır. Buna karşılık POST gibi yöntemlerde veri gövdeye de yazılabilir. Yani mesaj yapısını anlamak, method davranışını anlamının ön koşuludur.

Kısa Özet

HTTP istek mesajı; istek satırı, başlık satırları ve gerekirse gövdeden oluşan yapılandırılmış bir metindir. Method, URL ve version birlikte neyin, nereye ve hangi protokol kuralıyla istendiğini tanımlar.

Diğer HTTP İstek Mesajları

Bu slayt, GET’in tek HTTP method’u olmadığını ve farklı method’ların farklı kullanım amaçları taşıdığını gösterir. POST, kullanıcı tarafından girilen veriyi mesaj gövdesinde sunucuya taşımak için uygundur. GET ise normalde kaynak istemek için kullanılır; ancak bazı uygulamalar kullanıcı verisini URL sonuna ekleyerek GET ile de sunucuya parametre geçebilir. HEAD, gövde olmadan yalnızca başlıkları istemek için kullanılır. PUT ise belirli bir URL’deki kaynağı yüklemek veya bütünüyle değiştirmek amacıyla düşünülür.

Burada karıştırılabilecek nokta, GET ile POST’un yalnızca “ikisi de veri gönderiyor” düzeyinde düşünülmesidir. Aradaki fark önemlidir. GET’te parametrelerin URL içinde görünmesi, önbellekleme, günlükleme ve bağlantı paylaşımı gibi davranışları etkileyebilir. POST’ta ise veri gövdede taşınır; bu nedenle semantik olarak daha çok “sunucuya veri sunma” davranışıyla ilişkilidir. HEAD ise çoğu zaman kaynak hakkında bilgi almak, ama kaynağın kendisini indirmemek için kullanılır.

Bu slaytı ezberlenecek method listesi gibi değil, istemci niyetinin protokole nasıl yansıtıldığı olarak düşünmelisin. Aynı TCP bağlantısı üzerinde giden iki HTTP isteği, method değişince tamamen farklı davranış beklentisi oluşturabilir.

Kısa Özet

HTTP method'ları, istemcinin niyetini protokole açık biçimde yansıtır. GET, POST, HEAD ve PUT aynı mesaj yapısını paylaşıyor da semantik olarak farklı kullanım amaçlarına sahiptir.

HTTP Yanıt Mesajı

HTTP isteğine karşılık dönen mesaj da en az istek kadar yapılandırılmıştır. Yanıt mesajının başında durum satırı (status line) vardır; burada protokol sürümü, status code ve status phrase bulunur. Bunun altında header satırları, en altta da istenen nesnenin kendisini taşıyan entity body yer alır. Kitaptaki örnekte `HTTP/1.1 200 OK` satırı, hem kullanılan sürümü hem de isteğin başarıyla işlendiğini gösterir. `Content-Length`, `Content-Type`, `Last-Modified` ve `Connection` gibi alanlar yanıtın nasıl yorumlanacağını belirler.

Burada özellikle şunu anlamalısın: yanıt mesajının “esas yükü” çoğu zaman gövdedeki nesnedir; ama başlıklar bu nesnenin nasıl ele alınacağını belirler. `Content-Type` nesnenin türünü söyler, `Content-Length` uzunluğu verir, `Last-Modified` önbellekleme açısından kritik bilgi sağlar. Bu nedenle header alanları yardımcı bilgi değil, uygulama davranışını yöneten önemli sinyallerdir.

Status code kavramı da yalnızca teknik bir sayı değildir. `200 OK` başarıyı, `301 Moved Permanently` kalıcı yönlendirmeyi, benzer şekilde diğer kodlar da farklı işlem sonuçlarını anlatır. Bu, protokol semantiğinin pratikte nasıl işlendiğini gösterir: istemci yalnızca veri almaz, aynı zamanda sunucunun sonucu nasıl sınıflandırdığını da öğrenir.

Kısa Özet

HTTP yanıt mesajı; status line, header alanları ve entity body'den oluşur. Asıl nesne gövdede taşınsa da, başlıklar ve durum kodları istemcinin yanıtı nasıl yorumlayacağını belirler.

HTTP Yanıt Mesajı – Durum Kodlarının Mantığı

Durum kodlarını ezberlemekten çok, ne işe yaradıklarını anlamalısın. Status code, sunucunun istemcinin isteğine nasıl bir sonuç verdiğini özetleyen kısa bir sinyaldir. `200 OK`, istenen nesnenin bulunduğunu ve döndürüldüğünü gösterir. `301 Moved Permanently`, nesnenin yeni bir adrese taşındığını ve istemcinin bundan sonra yeni URL'yi kullanması gerektiğini anlatır. Bu nedenle durum kodları, uygulama katmanında kontrol akışının önemli bir parçasıdır.

Burada önemli olan, protokolün yalnızca “başarılı veri taşıma” değil, “uygun davranış yönlendirme” işi de yapmasıdır. Bazen istemciye veri gelir, bazen yeni adrese gitmesi söylenir, bazen nesnenin değişmediği bildirilir. Dolayısıyla status code mekanizması, istemci tarafında sonraki adımı belirleyen karar noktasıdır.

Bu kavram ileride koşullu GET ve önbellekleme ile birleştğinde daha da anlamlı hale gelir. Çünkü bazı yanıtlar veri gövdesi taşımadan bile çok değerli olabilir; önemli olan, istemciye ne yapması gerektiğini söylemesidir.

Kısa Özet

HTTP durum kodları, isteğin sonucunu kısa ama güçlü bir biçimde ifade eder. İstemcinin sonraki davranışı çoğu zaman bu kodlara göre belirlenir.

Kullanıcı/Sunucu Durumunun Korunması: Çerezler

HTTP stateless bir protokoldür; yani sunucu, varsayılan olarak önceki istekleri hatırlamak zorunda değildir. Bu sadelik ölçeklenebilirlik açısından büyük avantaj sağlar, ancak gerçek uygulamalarda kullanıcıyı tanımak çoğu zaman gerekir. İşte çerezler (cookies) bu boşluğu kapatır. Kitap ve slayt, çerez teknolojisinin dört temel bileşeni olduğunu vurgular: HTTP yanıtındaki `Set-Cookie` başlığı, sonraki isteklerdeki `Cookie` başlığı, istemci tarafında tarayıcı tarafından yönetilen çerez dosyası ve sunucu tarafındaki arka uç veritabanı (back-end database).

Bu yapıyı şöyle düşünebilirsin: Sunucu ilk ziyarette kullanıcıya benzersiz bir kimlik verir. Tarayıcı bu kimliği saklar. Sonraki isteklerde bu kimlik geri gönderilir. Sunucu da arka plandaki veritabanında bu kimliğe bağlı tercihleri, oturum bilgisini veya sepet durumunu bulabilir. Böylece stateless HTTP'nin üstünde, fiilen stateful bir kullanıcı deneyimi kurulmuş olur.

Burada dikkat edilmesi gereken nokta, çerezin tek başına “tüm durumu saklaması” gerekmediğidir. Çoğu zaman çerez, asıl veriye işaret eden bir anahtar gibi davranır. Asıl durum bilgisi sunucudaki veritabanında tutulur. Bu yüzden cookie mechanism, durumun hem istemci hem sunucu tarafında parçalı biçimde korunmasını sağlar.

Kısa Özet

Çerezler, stateless HTTP üzerinde kullanıcı durumunu izleyebilmek için kullanılan mekanizmadır. Temel mantık, istemciye bir kimlik verip sonraki isteklerde bu kimliği geri almaktır.

HTTP Çerezleri: Yorumlar

Çerezlerin kullanım alanı oldukça geniştir: yetkilendirme, alışveriş sepeti, öneri sistemleri ve kullanıcı oturum durumunun korunması bunların başlıcalarıdır. Ancak aynı mekanizma gizlilik açısından da dikkat gerektirir. Slaytta vurgulandığı gibi üçüncü taraf kalıcı çerezler (third-party persistent cookies), kullanıcı davranışının birden fazla site boyunca izlenmesine imkân verebilir. Bu yüzden çerezler yalnızca teknik bir kolaylık değil, aynı zamanda veri mahremiyeti tartışmasının merkezindeki araçlardan biridir.

Burada “state nasıl korunur?” sorusu iki farklı yaklaşım doğurur. Birincisi, istemci veya sunucu uçlarında çoklu etkileşim boyunca doğrudan durum saklamaktır. İkincisi ise durum

bilgisini mesajların içinde, örneğin cookie alanı üzerinden taşımaktır. Web mimarisinde çoğu zaman bu iki yaklaşım birlikte kullanılır.

Bu konuyu teknik ve etik boyutuyla birlikte düşünmelisin. Çerezler, web'i daha kişiselleştirilmiş ve kullanışlı hale getirir; fakat aynı zamanda kullanıcı takibi ve profil çıkarımı için de kullanılabilir. Modern tarayıcı politikaları ve düzenlemeler bu yüzden çerez kullanımını daha sıkı kontrol etmeye başlamıştır.

Kısa Özet

Çerezler, oturum ve kişiselleştirme için çok kullanışlıdır; ancak gizlilik açısından risk de taşır. Teknik olarak durum koruma sağlarlar, pratikte ise aynı zamanda izleme altyapısının bir parçası olabilirler.

Örnek: NY Times Web Sayfasını Görüntüleme

Bu görsel, bir web sayfasının yalnızca tek bir sunucudan gelen içerikten oluşmadığını göstermesi bakımından önemlidir. Kullanıcı temel HTML dosyasını örneğin `nytimes.com` sunucusundan alırken, sayfaya gömülü reklam gibi başka nesneler üçüncü taraf bir alan adından, örneğin bir reklam sunucusundan getirilebilir. Sonunda tarayıcı bu bileşenleri birleştirerek kullanıcıya tek bir sayfa gösterir.

Bu şemayı anlamak, modern web'in neden yalnızca “istemci bir sunucuya gider” kadar basit olmadığını fark ettirir. Tek bir kullanıcı etkileşimi, birden fazla HTTP isteği, birden fazla sunucu ve bazen birden fazla kurumsal taraf içerebilir. Bu da performans, önbellekleme, güvenlik ve gizlilik konularını doğrudan etkiler.

Burada özellikle üçüncü taraf içerik kavramına dikkat etmelisin. Sayfayı ziyaret ettiğinde yalnızca esas siteyle değil, gömülü nesneleri sağlayan başka sistemlerle de dolaylı olarak etkileşime girmiş olabilirsin. Bu, çerezlerin neden siteler arası izleme bağlamında tartışıldığını da açıklar.

Kısa Özet

Modern bir web sayfası çoğu zaman tek kaynaktan gelmez; temel içerik ve gömülü nesneler farklı sunuculardan çekilebilir. Bu yapı, web performansı ve gizlilik konularını daha karmaşık hale getirir.

Web Önbellekleri

Web önbelleği (web cache / proxy server), istemci ile origin server arasında yer alarak sık istenen nesneleri kullanıcıya daha yakın bir noktadan sunmaya çalışır. Slaytta açıkça belirtildiği gibi önbellek hem istemci hem de sunucu gibi davranır: istemci açısından bir sunucu gibi görünür; ama origin server'dan nesne isterken kendisi istemci rolüne geçer. Eğer nesne önbellekte varsa hemen döndürülür; yoksa origin server'dan alınır, önbelleğe konur ve ardından istemciye gönderilir.

Bu mekanizmanın üç temel avantajı vardır. Birincisi, yanıt süresini azaltır; çünkü önbellek kullanıcıya fiziksel ve topolojik olarak daha yakındır. İkincisi, kurumun erişim bağlantısındaki trafiği düşürür. Üçüncüsü, origin server üzerindeki yükü azaltır ve içeriğin daha verimli dağıtılmasını sağlar. Slayttaki “location is everything” vurgusu bu yüzden anlamlıdır: içerik kullanıcıya ne kadar yakınsa gecikme ve ağ maliyeti o kadar azalabilir.

Burada karıştırılmaması gereken nokta şudur: önbellek kopya üretir ama bu kopya “eski veri” sorununu da beraberinde getirebilir. Bu nedenle sonraki slaytlarda conditional GET ve Last-Modified gibi mekanizmalar devreye girer.

Kısa Özet

Web önbelleği, istemciye yakın noktada içerik saklayarak gecikmeyi ve ağ yükünü azaltır. Proxy gibi davranır; hem istemciye sunucu olur hem de origin server’a istemci gibi istek gönderir.

Web Önbellekleri (Diğer Adıyla Proxy Sunucuları)

Bu slayt, web önbelleğinin ağ içindeki rolünü daha netleştirir. Proxy sunucu, istemcinin taleplerini origin server’a taşımadan önce kendisi karşılayabiliyorsa büyük verim sağlar. Özellikle kurumsal ağlarda veya erişim bağlantısı dar boğaz olduğunda bu etki belirgindir. Slaytta, erişim linkindeki trafiği azaltmak ve istemci yanıt süresini kısaltmak temel motivasyon olarak verilir. Ayrıca sunucu tarafının yanıt başlıklarında nesnenin önbelleklenebilirliğine dair bilgi taşıyabileceği de belirtilir.

Burada dikkat edilmesi gereken şey, proxy’nin yalnızca “kopya saklama” işi yapmadığıdır. Aslında ağ kaynaklarının kullanım biçimini yeniden düzenler. Özellikle aynı içerik çok sayıda kullanıcı tarafından isteniyorsa, tek tek uzak sunucuya gitmek yerine yerel kopyayı döndürmek çok daha mantıklıdır.

Bu slayt ağ mimarisi açısından da önemlidir. Çünkü uygulama katmanı çözümü gibi görünse de, etkisi doğrudan bağlantı kullanımı, kuyruk gecikmesi ve maliyet üzerindedir. Yani web cache, uygulama katmanı ile ağ performansı arasında güçlü bir köprü kurar.

Kısa Özet

Proxy sunucu, istemci taleplerini yerelde karşılayarak hem gecikmeyi hem de dış ağ trafiğini azaltır. Özellikle aynı içeriğe tekrar tekrar erişilen ortamlarda çok etkili bir çözümdür.

Önbellekleme Örneği

Bu örnek, web önbelleğinin neden pratikte bu kadar önemli olduğunu sayısal olarak gösterir. Kurumsal ağ içindeki LAN çok hızlı olabilir; fakat dış dünyaya açılan erişim bağlantısı çok daha yavaş olabilir. Slaytta 1.54 Mbps erişim linki, 1 Gbps LAN ve origin server’a doğru 2 saniyelik RTT gibi değerlerle gösterilen senaryoda, erişim linki neredeyse doymuş hale gelir ve yüksek kuyruk gecikmeleri oluşur. Yani sorun içerik boyutundan çok, dar boğaz bağlantının aşırı kullanılmasıdır.

Burada anlamam gereken temel ilişki şudur: bağlantı kullanımı (utilization) kapasite sınırına yaklaştıkça kuyruk gecikmesi doğrusal değil, çok hızlı büyüyebilir. Bu yüzden “bağlantı hâlâ çalışıyor” olması iyi performans anlamına gelmez. Web performansının bozulması çoğu zaman doğrudan paket kaybindan değil, kuyrukta bekleme süresinin patlamasından kaynaklanır.

Bu örnek, neden “sadece daha hızlı link satın alalım” yaklaşımının tek çözüm olmadığını da zeminler. Çünkü bazen akıllı önbellekleme, çok daha düşük maliyetle büyük iyileşme sağlar.

Kısa Özet

Dar boğaz erişim bağlantısı yüksek kullanıma ulaştığında gecikme hızla büyür. Web önbellekleme, bu dar boğaza binen yükü azaltarak performansı ciddi biçimde iyileştirebilir.

Web Önbelleği ile Ortalama Gecikmenin Azaltılması

Slayttaki önbellek isabet oranı (cache hit rate) örneği, önbelleğin yalnızca “bazı nesneleri yerelden verme” değil, toplam ortalama gecikmeyi aşağı çekme gücünü açıkça gösterir. Örneğin taleplerin yüzde 40’ı önbellekten, düşük milisaniye gecikmesiyle karşılanıyorsa ve yalnızca yüzde 60’ı origin server’a gidiyorsa, erişim bağlantısı üzerindeki trafik ciddi biçimde azalır. Slaytta erişim linki kullanımı yaklaşık 0.58’e düşerken, ortalama uçtan uca gecikme de belirgin şekilde iyileşir.

Burada önemli olan, ortalama gecikmenin ağırlıklı bir bileşim olarak düşünülmesidir. Önbellekten karşılanan istekler çok ucuzdur; origin server’a gidenler daha pahalıdır. Hit rate arttıkça pahalı yol daha az kullanılır. Dolayısıyla önbellek, performansı yalnızca tekil istek düzeyinde değil, sistem ortalaması düzeyinde iyileştirir.

Bu slayt sana şunu öğretmelidir: performans çözümü her zaman daha pahalı fiziksel kapasite almak değildir. Bazen daha akıllı içerik yerleştirme ve tekrar kullanılan nesneleri yakında tutma, daha ucuz ve daha etkili bir çözümdür.

Kısa Özet

Önbellek isabet oranı yükseldikçe dış bağlantı yükü ve ortalama gecikme düşer. Web önbellekleme, performansı maliyet-etkin biçimde iyileştiren güçlü bir uygulama katmanı stratejisidir.

Tarayıcı Önbelleği: Koşullu GET

Koşullu GET (conditional GET), hem gecikmeyi hem de gereksiz veri aktarımını azaltmak için kullanılan çok akıllı bir HTTP mekanizmasıdır. Temel fikir şudur: istemcide nesnenin bir kopyası zaten varsa, ama güncel olup olmadığından emin değilse, sunucuya “eğer şu tarihten sonra değiştiyse gönder” anlamına gelen bir istek yollar. Bunun için If-Modified-Since başlığı kullanılır. Eğer sunucudaki nesne değişmemişse, sunucu 304 Not Modified döner ve nesneyi yeniden göndermek zorunda kalmaz. Eğer değişmişse normal 200 OK yanıtı ve güncel veri döner.

Bu mekanizma çok önemlidir; çünkü önbellekleme her zaman “yerelde varsa kullan” kadar basit değildir. Asıl mesele, yereldeki kopyanın güncel kalıp kalmadığını ekonomik biçimde test etmektir. Conditional GET tam da bunu yapar. Nesne değişmemişse, ağ kaynakları boşa harcanmaz. Yani burada veri yerine doğrulama taşınır.

Bu slaytı anlamak, HTTP’nin sadece dosya indirip gönderen kaba bir protokol olmadığını; önbellek, doğrulama ve bant genişliği tasarrufu gibi daha olgun davranışlar taşıdığını gösterir.

Kısa Özet

Koşullu GET, istemcinin elindeki kopya güncelse nesnenin yeniden gönderilmesini önler. If-Modified-Since ve 304 Not Modified birlikte, gecikmeyi ve ağ yükünü azaltan zarif bir doğrulama mekanizması oluşturur.

Uygulama Katmanı: Genel Bakış – E-posta, SMTP, IMAP’e Geçiş

Bu yol haritası slaytı, Web ve HTTP konusundan e-posta mimarisine geçtiğini gösterir. Bu geçiş öğretici açıdan önemlidir; çünkü her ikisi de uygulama katmanı protokolüdür, ama çalışma tarzları farklıdır. Web çoğunlukla istemci-sunucu nesne isteme modeliyle çalışırken, e-posta store-and-forward mantığına, sunucu kuyruklarına ve ayrı alma/gönderme protokollerine dayanır.

Burada önceki konularla bağ kurmalısın. Yine süreçler konuşacak, yine taşıma protokolü seçilecek, yine mesaj biçimi tanımlanacak. Ama bu kez etkileşim tek bir anlık istek-yanıt akışı olmaktan çıkıp, kullanıcı aracı, posta sunucusu, kuyruk ve alma protokolü içeren daha katmanlı bir işleyişe dönüşüyor.

Kısa Özet

E-posta bölümü, uygulama katmanı ilkelerini farklı bir uygulama ailesi üzerinde yeniden düşünmeni sağlar. Burada Web’den farklı olarak iletme, depolama ve sonradan alma mantığı öne çıkar.

E-posta

E-posta mimarisi üç temel bileşene dayanır: user agents, mail servers ve SMTP. User agent, kullanıcının mesaj yazdığı, okuduğu ve düzenlediği arayüzdür; Outlook veya telefon üzerindeki posta istemcisi bunun örneğidir. Mail server, gelen mesajları mailbox içinde, gidecek mesajları ise outgoing message queue içinde tutar. SMTP ise posta sunucuları arasında mesaj iletimini sağlayan temel protokoldür. Kitap da e-posta sistemini bu üçlü yapı üzerinden açıklar.

Burada dikkat edilmesi gereken nokta, e-postanın “gönder ve anında karşı tarafa ulaştı” gibi doğrudan bir modelle çalışmamasıdır. Kullanıcı mesajı önce kendi posta sunucusuna verir. Mesaj burada kuyruklanabilir. Sonra gönderici posta sunucusu, alıcı posta sunucusuna SMTP ile aktarım yapar. Alıcı kullanıcı ise mesajı daha sonra kendi user agent’ı üzerinden alır veya okur.

Bu mimari, e-postanın neden asenkron bir iletişim biçimi olduğunu açıklar. Web’de istemci çoğu zaman yanıtı hemen bekler; e-postada ise iletim ve alma farklı zamanlarda gerçekleşebilir.

Kısa Özet

E-posta sistemi; kullanıcı arayüzü, posta sunucusu ve SMTP protokolü üzerine kuruludur. Temel mantık anlık istek-yanıt değil, iletme, saklama ve sonradan erişim modelidir.

E-posta: Posta Sunucuları

Posta sunucusunun iki temel deposu vardır: mailbox ve message queue. Mailbox, kullanıcıya gelen mesajların saklandığı yerdir. Message queue ise gönderilmeyi bekleyen iletileri tutar. SMTP, bu sunucular arasında mesaj taşır; yani bir sunucu başka bir sunucuya ileti gönderirken istemci gibi, ileti alırken sunucu gibi davranabilir. Kitap da özellikle bunu vurgular: SMTP her mail server üzerinde bulunur; başka sunuculara gönderirken SMTP client, alırken SMTP server rolündedir.

Bu ayrımı doğru kurman gerekir: kullanıcı doğrudan diğer kullanıcının makinesine değil, önce kendi posta sunucusuna konuşur. Ardından sunucular kendi aralarında SMTP ile çalışır. Bu da sistemin neden dayanıklı ve zamandan bağımsız çalışabildiğini gösterir. Alıcı sunucu geçici olarak erişilemezse, mesaj gönderen tarafta kuyrukta bekleyebilir.

Bu yapı store-and-forward mantığının özüdür. Mesaj, yol boyunca geçici depolama ve yeniden deneme olanağına sahiptir. Bu yüzden e-posta, etkileşimli değil ama dayanıklı bir iletişim modelidir.

Kısa Özet

Posta sunucusu, gelen mesajları mailbox’ta, gönderilecek mesajları message queue’da tutar. Sunucular arası iletim SMTP ile yapılır ve sistem store-and-forward mantığıyla çalışır.

SMTP RFC 5321

SMTP, Internet e-postasının kalbindeki protokoldür ve TCP üzerinde, tipik olarak port 25 kullanarak güvenilir aktarım yapar. Slaytta transferin üç aşaması açıkça verilir: SMTP handshaking, message transfer ve closure. Ayrıca SMTP’nin komut/yanıt etkileşimi bakımından HTTP’ye benzediği, fakat e-posta iletimi için tasarlandığı vurgulanır. Komutlar ASCII metindir, yanıtlar ise durum kodu ve açıklama içerir. Kitap da Alice ve Bob örneği üzerinden bu akışı adım adım anlatır.

Burada önemli olan, SMTP’nin doğrudan aktarım modeline sahip olmasıdır. Gönderen sunucu, alıcı sunucuya doğrudan TCP bağlantısı açar. Eğer alıcı sunucu o anda kapalıysa mesaj bir ara sunucuya bırakılmaz; gönderen tarafta kuyrukta bekler ve daha sonra yeniden denir. Bu, web’den oldukça farklı bir davranıştır.

Bir başka önemli nokta da şudur: SMTP eski bir protokoldür ve bazı tarihsel kısıtlamalar taşır. Kitap, mesaj gövdesinin 7-bit ASCII ile sınırlı olmasını “legacy” bir özellik olarak açıklar. Bu yüzden modern ekler ve çoklu ortam içerikleri için ek kodlama mekanizmaları gerekir.

Kısa Özet

SMTP, posta sunucuları arasında e-posta aktaran, TCP tabanlı ve komut/yanıt yapısına sahip bir protokoldür. E-posta iletiminde store-and-forward ve doğrudan sunucudan sunucuya aktarım mantığı öne çıkar.

SMTP RFC (5321)

SMTP, İnternet e-postasının sunucudan sunucuya aktarımını yapan temel uygulama katmanı protokolüdür. Burada dikkat edilmesi gereken nokta, SMTP’nin kullanıcı ile sunucu arasında değil, çoğu zaman gönderici posta sunucusu ile alıcı posta sunucusu arasında çalışmasıdır. Bu protokol TCP kullanır ve tipik olarak 25 numaralı port üzerinden güvenilir iletim sağlar. Yani SMTP kendi başına güvenilirlik üretmek zorunda kalmaz; bu işi TCP’den alır.

Slayttaki üç aşamalı yapı çok önemlidir: önce SMTP handshaking yapılır, sonra mesaj aktarılır, en sonda bağlantı kapatılır. Handshake kısmı yalnızca “selamlaşma” değildir; iki sunucunun birbirini tanıması ve gönderici ile alıcı adres bağlamının kurulması açısından gereklidir. Bu yüzden HELO gibi komutlar, yüzeyde basit görünse de uygulama katmanı düzeyinde iletişim oturumunu başlatan işaretlerdir. Sunucu yanıtları da yalnızca metin değil, durum kodu + açıklama biçimindedir. Bu da SMTP’nin, HTTP gibi, komut/yanıt temelli bir protokol olduğunu gösterir.

Bu kavramı şöyle düşünebilirsin: HTTP’de istemci bir nesneyi “çekmek” için istek yapar; SMTP’de ise gönderici taraf, mesajı karşı tarafa “itmek” için süreci başlatır. Bu yüzden e-posta tarafında aktarım mantığı, web’den biraz daha teslimat odaklıdır.

Kısa Özet

SMTP, e-postayı posta sunucuları arasında TCP üzerinden güvenilir biçimde taşıyan temel protokoldür. El sıkışma, mesaj aktarımı ve bağlantı kapatma aşamaları birlikte, e-postanın sunucudan sunucuya iletim mantığını oluşturur.

Senaryo: Alice, Bob’a E-posta Gönderir

Bu slayt, e-posta iletim sürecini adım adım gösterdiği için çok öğreticidir. İlk adımda Alice, user agent üzerinden Bob’a bir mesaj yazar. Sonra bu mesaj Alice’in posta sunucusuna iletilir ve giden mesaj kuyruğuna konur. Ardından Alice’in posta sunucusundaki SMTP istemcisi, Bob’un posta sunucusuna TCP bağlantısı açar ve mesajı gönderir. Son aşamada Bob’un sunucusu mesajı Bob’un mailbox’ına bırakır; Bob da istediği zaman kendi user agent’ı ile bu mesajı okur.

Burada çok önemli olan, e-postanın asenkron bir iletişim modeli olmasıdır. Yani Bob’un o anda çevrim içi olması gerekmez. Hatta Bob’un user agent’ı açık olmasa bile sorun değildir; önemli olan Bob’un posta sunucusunun mesajı kabul edebilmesidir. Bu, e-postayı web’den

ayırır temel özelliklerden biridir. Web’de çoğu zaman istemci yanıtı o anda bekler; e-postada ise ileti önce teslim edilir, sonra uygun zamanda okunur.

Bir başka kritik nokta da şudur: SMTP genellikle araya keyfi ara sunucular koyarak çalışmaz. Kitaptaki örnek bunu açıkça anlatır. Gönderen sunucu, alıcı sunucuya doğrudan bağlanmaya çalışır. Eğer alıcı sunucu geçici olarak erişilemez durumdaysa, mesaj ara bir sunucuya “emanet” edilmez; gönderen tarafta kuyrukta bekler ve daha sonra yeniden denir.

Kısa Özet

E-posta iletimi, user agent → gönderici posta sunucusu → alıcı posta sunucusu → alıcı user agent akışıyla işler. Bu model asenkron; mesaj hemen okunmak zorunda değildir, önce güvenli biçimde sunucu tarafında saklanır.

Örnek SMTP Etkileşimi

Bu slayt, SMTP’nin soyut tanımını somut bir protokol diyaloguna dönüştürüyor. 220, 250, 354, 221 gibi kodlarla başlayan sunucu yanıtları ve HELO, MAIL FROM, RCPT TO, DATA, QUIT gibi istemci komutları, uygulama katmanı protokollerinin aslında ne kadar açık ve kural tabanlı çalıştığını gösterir. Burada özellikle MAIL FROM göndericiyi, RCPT TO alıcıyı, DATA ise artık mesaj gövdesinin geleceğini belirtir. Mesajın sonu da tek başına bir satırdaki nokta ile, yani CRLF.CRLF mantığıyla işaretlenir.

Bu örneği incelerken şunu fark etmelisin: SMTP’nin mesajın “kimden geldiği” ve “kime gideceği” bilgisini, e-posta gövdesinin parçası olarak değil, protokol diyalogunun bir parçası olarak taşıması çok önemlidir. Yani henüz mail body gelmeden önce bile sunucu, iletimin bağlamını biliyor olur. Bu da sunucunun kabul, ret veya yönlendirme gibi kararları erken vermesine izin verir.

Ayrıca bu diyalog, telnet ile elle de denenebilecek kadar açık metin tabanlıdır. Bu durum öğretici açıdan avantajlıdır; çünkü protokolün davranışı gözle görülebilir hale gelir. Fakat aynı açıklık, güvenlik eklenmediğinde açık metin taşımanın risklerini de hatırlatır.

Kısa Özet

SMTP etkileşimi, istemci komutları ve sunucu durum kodları üzerinden yürür. Gönderici, alıcı ve mesaj aktarımı açık bir sırayla tanımlandığı için protokolün mantığı şeffaf ve izlenebilirdir.

SMTP: Gözlemler

Bu slayt, SMTP’nin temel davranışını HTTP ile karşılaştırarak netleştirir. SMTP kalıcı bağlantı kullanabilir; yani aynı TCP bağlantısı üzerinden birden fazla mesaj gönderilebilir. Bunun yanında, SMTP tarihsel olarak mesajın başlık ve gövdesini 7-bit ASCII biçiminde taşımaya dayanır. Kitap da bunu özellikle “legacy” bir özellik olarak anlatır. Bu sınırlama, modern çoklu ortam içeriklerinin önce ASCII’ye kodlanmasını, sonra karşı tarafta tekrar çözülmesini gerektirir. HTTP’de ise bu tür bir zorunluluk yoktur.

Bir diğerk önemli fark, istemci davranışının yönüdür. HTTP’de istemci tipik olarak bir nesneyi “pull” eder; yani ister ve sunucu gönderir. SMTP’de ise gönderici taraf mesajı “push” eder; yani alıcı sunucuya doğru aktarmaya çalışır. Her iki protokol de ASCII tabanlı komut/yanıt yapısına sahip olsa da, etkileşim felsefeleri farklıdır. HTTP nesne isteme odaklıdır; SMTP teslim etme odaklıdır.

Burada özellikle `CRLF.CRLF` sonlandırma kuralını iyi anlamalısın. Çünkü SMTP, mesaj gövdesinin bittiğini içerik boyundan değil, bu özel sonlandırma işaretinden anlar. Bu da protokolün metin tabanlı tarihsel tasarımını yansıtır.

Kısa Özet

SMTP ile HTTP benzer biçimde komut/yanıt yapısı kullansa da, biri push diğeri pull mantığıyla çalışır. SMTP’nin 7-bit ASCII ve `CRLF.CRLF` gibi tarihsel özellikleri, onun eski ama hâlâ çok yaygın bir protokol olduğunu gösterir.

E-posta Mesajı Formatı

Bu slaytta kritik ayrım şudur: SMTP, e-posta mesajlarının nasıl taşınacağını tanımlar; ama e-posta mesajının kendi iç sözdizimini ayrı bir standartla tanımlanır. Sunuda RFC 2822 denmiş, kitapta modern çerçeve RFC 5322 olarak anlatılır; temel fikir değişmez. Mesajın kendisi başlık (header) ve gövde (body) kısımlarından oluşur ve arada boş bir satır bulunur. `To:`, `From:`, `Subject:` gibi alanlar mesajın parçasıdır; bunlar SMTP el sıkışmasındaki `MAIL FROM` ve `RCPT TO` komutlarıyla aynı şey değildir. Bu ayrımı mutlaka net kuralısın.

Bu ayrım neden önemlidir? Çünkü SMTP komutları taşıma sürecinin kontrol bilgisidir; mesaj başlığı ise alıcıya gösterilecek, arşivlenecek ve istemci tarafından yorumlanacak içerik bilgisidir. Yani ikisi hem görünüşte benzer kelimeler kullanır hem de tamamen farklı katman rollerine sahiptir.

Bu kavramı şöyle düşünebilirsin: Kargo şirketine “paket kimden, kime gidiyor?” diye verilen teslimat bilgisiyle, paketin içine konan mektubun üst bilgisi aynı şey değildir. SMTP komutları birincisine, e-posta başlık alanları ikincisine benzer.

Kısa Özet

E-posta mesaj formatı ile SMTP komutları farklı şeylerdir. SMTP iletim kurallarını belirler; mesaj formatı ise `From`, `To`, `Subject` ve `body` gibi alanlarla e-postanın kendi yapısını tanımlar.

E-posta Alma: Posta Erişim Protokolleri

SMTP gönderme tarafını çözer; fakat alıcının mesajı kendi posta sunucusundan nasıl alacağını ayrı bir problemdir. İşte burada posta erişim protokolleri devreye girer. Slayt, alıcı tarafında IMAP veya HTTP gibi erişim protokollerinin kullanılabildiğini gösterir. Kitapta da POP3, IMAP ve web tabanlı erişim ayrı yaklaşımlar olarak ele alınır. Ana fikir şudur: bir e-posta mesajının sunucular arasında aktarılması ile kullanıcının kendi posta kutusuna erişmesi iki farklı protokol ailesiyle yönetilir.

Burada neden ayrı bir erişim protokolüne ihtiyaç duyulduğunu iyi anlamalısın. Bob'un kendi bilgisayarında sürekli açık bir posta sunucusu yoktur. Mesajlar önce Bob'un mail server'ındaki mailbox'a gelir. Sonra Bob, kendi cihazından bu kutuya bağlanarak iletileri alır, listeler, silebilir veya sunucuda bırakabilir. Yani alım işlemi, sunucudan kullanıcıya doğru ikinci bir erişim sürecidir.

Bu ayrım, e-posta sisteminin neden çok katmanlı ve dayanıklı olduğunu gösterir. Gönderme, sunucular arası taşıma ve son kullanıcı erişimi birbirinden ayrıldığı için sistem esnek hale gelir.

Kısa Özet

SMTP mesajı posta sunucularına kadar taşır; alıcının mesajı okuması için ise ayrıca bir posta erişim protokolü gerekir. Gönderme ve alma, e-posta sisteminde farklı protokol sorunlarıdır.

POP3

POP3, en sade posta erişim protokollerinden biridir. Temel mantığı, alıcının posta sunucusuna bağlanıp mesajları almasıdır. Bu protokol üç evreyle düşünülür: yetkilendirme (authorization), işlem (transaction) ve güncelleme (update). İlk aşamada kullanıcı adı ve parola gönderilir. Sonra istemci gelen mesajları listelebilir, indirebilir, silinmek üzere işaretleyebilir. Son aşamada oturum kapanırken bu değişiklikler uygulanır. POP3'ün güçlü yanı sadeliğidir; fakat bu sadelik, esneklikten feragat edilerek elde edilir.

Burada önemli olan, POP3'ün çoğu zaman “download-and-delete” veya en azından “sunucudan çekip yerelde yönet” mantığıyla ilişkilendirilmesidir. Bu, tek cihazlı kullanımda pratik olabilir; fakat birden fazla cihazdan aynı posta kutusunu senkron biçimde yönetmek gerektiğinde sınırlayıcı hale gelir.

Bu yüzden POP3'ü “eski ve kötü” diye değil, “basit ve belirli senaryolar için uygun” diye düşünmek daha doğrudur. Tasarım felsefesi, sunucuda zengin klasör ve durum yönetiminden çok, istemciye mesaj teslim etmeye odaklıdır.

Kısa Özet

POP3, e-postayı basit biçimde sunucudan istemciye taşımaya odaklanan sade bir erişim protokolüdür. Gücü sadeliğinden gelir; ancak çok cihazlı ve durum senkronizasyonu gereken kullanımda sınırlamaları vardır.

IMAP

IMAP, POP3'e göre daha zengin ve sunucu-merkezli bir posta erişim modelidir. Temel fark, mesajların sunucuda tutulmaya devam etmesi ve kullanıcının posta kutusu durumunu sunucu tarafında yönetebilmesidir. Klasör yapıları, okunmuş/okunmamış durumu, sunucu üstünde arama ve farklı cihazlardan tutarlı görünüm sağlama gibi yetenekler bu yüzden IMAP ile daha doğal hale gelir. Sunudaki “IMAP” vurgusu da bu daha gelişmiş erişim modeline işaret eder.

Burada dikkat edilmesi gereken nokta, IMAP'in yalnızca "mesajı almak" için değil, posta kutusunu uzaktan yönetmek için tasarlanmış olmasıdır. Bu nedenle modern akıllı telefon, dizüstü ve webmail kombinasyonlarında IMAP tarzı yaklaşım çok daha uygun görünür. Mesaj bir cihazda okununca diğerinde de okunmuş görünmesi gibi davranışlar, bu sunucu-merkezli model sayesinde kolaylaşır.

Dolayısıyla POP3 ile IMAP farkını "ikisi de mail alıyor" diye küçümsememelisin. Aradaki fark, istemci ile sunucu arasında durumun nerede tutulduğuna dayanır.

Kısa Özet

IMAP, e-postayı yalnızca indirmekten çok, sunucu üzerindeki posta kutusunu uzaktan yönetmeye odaklanır. Bu yüzden çok cihazlı ve senkron kullanımda POP3'e göre daha güçlüdür.

DNS'e Giriş

DNS, İnternet'in dizin hizmetidir. İnsanlar alan adlarıyla düşünür; ağ ise IP adresleriyle çalışır. DNS'in temel görevi, bu iki dünya arasında çeviri yapmaktır. Ancak burada dikkat edilmesi gereken nokta, DNS'in yalnızca "hostname → IP" dönüştüren basit bir tablo olmadığıdır. Kitap, DNS'i dağıtık ve hiyerarşik bir veritabanı sistemi olarak tanımlar. Ölçek sorunu nedeniyle merkezi tek bir sunucu çözümü işe yaramaz; bu yüzden DNS tasarımı baştan dağıtık yapılmıştır.

Bu kavramı doğru kurman gerekir. Eğer tüm İnternet için tek bir merkezi ad sunucusu olsaydı, veritabanı devasa olurdu, güncellemeler zorlaşırdı ve tek hata noktası oluşurdu. DNS'in başarısı, dağıtıklık ve hiyerarşi sayesinde sağlanır. Bu açıdan DNS, yalnızca bir protokol değil, büyük ölçekli dağıtık sistem tasarımının da çok iyi bir örneğidir.

DNS'i anlamak, ağ mimarisindeki "isim" ile "adres" arasındaki ayrımı anlamaktır. Kullanıcı `www.example.com` ister; ağ sonunda bir IP ile çalışır. Bu dönüşüm görünmez biçimde ama sürekli yapılır.

Kısa Özet

DNS, alan adlarını IP adreslerine dönüştüren dağıtık ve hiyerarşik bir İnternet dizin hizmetidir. Onu basit bir tablo gibi değil, büyük ölçekli dağıtık bir veritabanı sistemi gibi düşünmelisin.

Üst Düzey Etki Alanı ve Yetkili Sunucular

Bu slayt, DNS hiyerarşisinin üst katmanlarını netleştirir. Root sunucular, TLD sunucular ve authoritative DNS sunucuları birlikte çalışır. TLD sunucular `.com`, `.org`, `.edu` gibi üst düzey alanlar ile ülke kodu alanlarından sorumludur. Authoritative sunucular ise belirli bir kuruluşun alan adı altındaki gerçek eşlemeleri, yani host adı ile IP adresi ilişkilerini taşır. Kitaptaki hiyerarşi şekli bunu açık biçimde gösterir.

Burada önemli olan, bu sunucuların aynı işi yapmadığını fark etmektir. Root sunucular nihai cevap vermez; çoğu zaman “hangi TLD’ye gitmen gerektiğini” söyler. TLD sunucular da çoğu zaman “hangi authoritative sunucuya gitmen gerektiğini” söyler. Asıl nihai eşleme çoğunlukla authoritative sunucudur. Yani çözümleme yukarıdan aşağıya yönlendirilerek ilerler.

Bu yapıyı bir bürokratik yönlendirme sistemi gibi düşünebilirsin: üst makam seni ilgili birime, o birim seni nihai yetkili masaya yönlendirir. DNS bu yönlendirme zincirini çok verimli biçimde kurar.

Kısa Özet

DNS hiyerarşisinde root, TLD ve authoritative sunucular farklı roller üstlenir. Root ve TLD çoğu zaman yönlendirir; nihai adrese karşılık gelen gerçek kayıt authoritative sunucuda bulunur.

Yerel DNS Ad Sunucuları

Yerel DNS sunucusu, teknik olarak DNS hiyerarşisinin resmî bir katmanı değildir; ama pratikte sistemin merkezindeki aktördür. Bir host isim çözümleme istediğinde sorgu önce genellikle yerel DNS sunucusuna gider. Bu sunucu yanıtı önbellekten verebilir ya da DNS hiyerarşisine sorgu iletebilir. Kitap da local DNS server’ı hiyerarşinin parçası olmayan ama mimarının merkezinde yer alan bir proxy gibi tarif eder. ISP’ler kullanıcılara yerel DNS sunucularını genellikle DHCP yoluyla bildirir.

Burada özellikle şunu anlamalısın: son kullanıcı çoğu zaman root veya TLD sunucularıyla doğrudan konuşmaz. Yerel DNS sunucusu kullanıcı adına bu işi yapar. Böylece önbellekleme etkili olur, tekrar eden sorgular hızlanır ve istemci tarafı daha basit kalır.

Bu yapı, recursive ve iterative çözümleme tartışmasını da anlamlı kılar. İstemci, çoğu zaman yerel DNS’e “benim için çöz” der. Yerel DNS de gerekirse hiyerarşi içinde dolaşır.

Kısa Özet

Yerel DNS sunucusu, kullanıcının isim çözümleme için başvurduğu ilk yerdir. Hiyerarşinin resmî parçası olmasa da önbellek ve aracı rolü nedeniyle DNS mimarisinde çok kritik bir konumdadır.

DNS Ad Çözümleme: Yinelenen Sorgu

Yinelenen sorgu (iterative query) mantığında, sorulan sunucu tam çözümü yapmak zorunda değildir; yalnızca bir sonraki gidilecek yeri söyleyebilir. Slayttaki örnekte `engineering.nyu.edu` içindeki bir host, `gaia.cs.umass.edu` için IP adresi ister. Yerel DNS sunucusu root’a gider, root TLD’yi gösterir, TLD authoritative sunucuyu gösterir, authoritative sunucu ise nihai cevabı döner. Yani her adımda “bu cevabı ben bilmiyorum ama şuraya sor” mantığı vardır.

Bu mekanizmanın avantajı, üst seviyedeki sunucuların tüm çözüm yükünü taşımamasıdır. Root sunucular, tüm sorguları uçtan uca çözmeye kalksaydı üzerlerindeki yük çok ağır olurdu. Bu yüzden gerçek dünyada root ve TLD sunucular iterative mantıkla çalışır. Bu, ölçeklenebilirlik açısından çok mantıklıdır.

Burada çizimdeki numaralı akışı yalnızca bir prosedür gibi değil, dağıtık sistemde sorumluluk paylaşımı gibi görmelisin. Her sunucu kendi bildiği kadar cevap verir ve zinciri ilerletir.

Kısa Özet

Yinelenen sorguda her DNS sunucusu tam cevabı vermek zorunda değildir; çoğu zaman bir sonraki gidilecek sunucuyu söyler. Bu yaklaşım, çözüm yükünü DNS hiyerarşisine dağıtır.

DNS Ad Çözümleme: Özyinelemeli Sorgu

Özyinelemeli sorgu (recursive query) mantığında ise sorguyu alan sunucu, çözüm işini istemci adına üstlenir. Slaytta bunun özellikle yerel DNS sunucusu bağlamında düşünülmesi gerektiği vurgulanır. Gerçek dünyada istemci → yerel DNS ilişkisi çoğu zaman recursive'dir; ama root ve TLD sunucular genellikle recursive çözüm yapmaz, iterative çalışır. Bu yüzden pratik sistem hibrit bir model kullanır: istemci yerel DNS'e recursive sorar, yerel DNS de hiyerarşi içinde iterative dolaşır.

Burada önemli olan recursive ve iterative ayrımını “hangi taraf iş yükünü taşıyor?” sorusuyla anlamandır. Recursive modelde yük daha çok sorguyu alan sunucuya biner. Iterative modelde ise sorgu yönlendirilerek adım adım ilerler.

Bu nedenle slayttaki “hiyerarşinin üst kademelerinde ağır yük?” sorusu yerindedir. Tüm sunucular recursive çalışsaydı DNS altyapısının ölçeklenebilirliği daha sorunlu olurdu. Hibrit model bunun dengeli çözümüdür.

Kısa Özet

Recursive sorguda çözüm yükü sorguyu alan sunucuya biner; iterative sorguda ise yük yönlendirmeye hiyerarşiye dağıtılır. Pratik DNS kullanımı çoğu zaman istemci için recursive, hiyerarşi içinde iterative bir hibrit modeldir.

DNS Bilgilerini Önbelleğe Alma

DNS önbellekleme, sistemin performansını ciddi biçimde iyileştirir. Bir ad sunucusu bir eşlemeyi öğrendiğinde, bu bilgiyi bir süre tutabilir ve sonraki sorgulara doğrudan önbellekten yanıt verebilir. Bu durum yanıt süresini düşürür, tekrar eden sorguların hiyerarşi boyunca dolaşmasını önler ve root/TLD üzerindeki yükü azaltır. Özellikle TLD sunucularına ilişkin bilgiler çoğu yerel DNS sunucusunda sıkça önbelleğe alınır.

Ancak burada kritik kavram TTL'dir, yani time to live. Her kayıt sonsuza kadar geçerli kalmaz. TTL dolduğunda önbellek girdisi atılır veya yeniden doğrulanması gerekir. Bu mekanizma, önbellekleme ile güncellik arasında denge kurar. Fakat tamamen kusursuz

değildir. Bir host'un IP adresi değişirse, eski bilgi TTL süresi boyunca bazı yerlerde yaşamaya devam edebilir.

Bu yüzden DNS çevirisini “best effort name-address translation” olarak görmek daha doğrudur. Sistem çok iyi çalışır; ama küresel önbelleklerin tamamen anlık ve kusursuz güncel olması beklenmez.

Kısa Özet

DNS önbellekleme yanıt süresini azaltır ve sorgu yükünü düşürür. Ancak TTL nedeniyle önbellek kayıtları geçici olarak eski bilgi içerebilir; bu yüzden hız ile güncellik arasında doğal bir denge vardır.

DNS Kayıtları

DNS veritabanının temel yapı taşı resource record, yani RR'dir. Slaytta bunun (`name`, `value`, `type`, `tTL`) biçiminde tutulduğu gösterilir. Burada `type` alanı kaydın anlamını belirler. `A` kaydı bir host adını IP adresine bağlar. `NS` kaydı bir domain için yetkili ad sunucusunu gösterir. `CNAME`, bir takma adı (alias) gerçek kanonik isme yönlendirir. `MX` ise bir alan adıyla ilişkili posta sunucusunu tanımlar. Kitap da bu dört temel kayıt tipini ayrıntılı biçimde açıklar.

Burada asıl öğrenmen gereken şey, DNS'in yalnızca `A` kaydı deposu olmadığıdır. DNS aynı zamanda yönlendirme, takma ad çözümü ve posta teslim altyapısı için de kritik bilgi taşır. Örneğin bir e-posta teslim edilirken `MX` kaydı devreye girer; web alias yapılarında `CNAME` önem kazanır.

Özellikle `NS` kaydı, DNS hiyerarşisinin çalışmasında kilit rol oynar. Çünkü iterative çözümleme sırasında “bir sonraki kime sorulacak?” sorusunun cevabı çoğu zaman `NS` kayıtları üzerinden verilir.

Kısa Özet

DNS kayıtları farklı amaçlara hizmet eder: `A` adres eşler, `NS` yetkili sunucuyu gösterir, `CNAME` takma ad çözer, `MX` posta sunucusunu tanımlar. DNS bu yüzden yalnızca isimden IP'ye dönüşüm yapan basit bir tablo değildir.

DNS Protokol Mesajları

DNS'te yalnızca iki tür mesaj vardır: sorgu (query) ve yanıt (reply). Ancak ilginç olan, her ikisinin de aynı temel mesaj biçimini kullanmasıdır. Başlık bölümünde identification alanı istemci ile yanıtı eşleştirmek için kullanılır. Flags alanında `qr`, `rd`, `ra`, `aa` gibi bitler bulunur. `RD` istemcinin recursive çözüm istediğini, `RA` sunucunun recursive hizmet verebildiğini, `AA` ise yanıtın authoritative olduğunu gösterir. Mesajın devamında question, answer, authority ve additional bölümleri yer alır.

Burada dikkat edilmesi gereken nokta, DNS mesajının yalnızca “sorulan isim ve verilen IP”den ibaret olmamasıdır. `Authority` bölümü, hangi sunucunun yetkili olduğunu

gösterebilir. Additional bölümü ise örneğin NS kaydında adı verilen sunucunun IP'sini ekleyerek sorguyu hızlandırabilir. Bu ek bilgi sayesinde istemci veya yerel DNS sunucusu daha az adımda ilerleyebilir.

Bu slaytı anlamak, protokol tasarımı mesaj başlığının ne kadar kritik olduğunu gösterir. Küçük bit alanları, tüm çözümleme davranışını değiştirebilir.

Kısa Özet

DNS sorgu ve yanıtları aynı mesaj yapısını kullanır. Başlık bitleri ve answer/authority/additional bölümleri, yalnızca sonucu değil çözümleme bağlamını da taşır.

DNS'e Bilgi Nasıl Eklenir?

Bu slayt, DNS'in yalnızca sorgulanan bir sistem değil, aynı zamanda yönetilen bir kayıt altyapısı olduğunu gösterir. Yeni bir alan adı kurulduğunda ilk adım registrar üzerinden domain kaydı yapmaktır. Ardından primary ve secondary authoritative name server bilgileri ve bunların IP adresleri kayıt kuruluşuna verilir. Bu bilgiler TLD düzeyine işlenir ve böylece İnternet'in geri kalanı bu alan adını bulabilir hale gelir.

Bir sonraki adımda registrar, ilgili TLD sunucularına NS kayıtları ekler. Ayrıca glue record olarak bilinen A kayıtları da gerekebilir. Bunun nedeni şudur: eğer TLD sunucusu yalnızca “bu alan adı için yetkili sunucu dns1.networkutopia.com” derse ama bu sunucunun IP'si bilinmiyorsa çözümleme döngüye girebilir. Glue record, bu düğümü çözer ve authoritative sunucuya ulaşmayı mümkün kılar. Slayttaki örnek bunu çok güzel gösteriyor.

Burada önemli olan, DNS yönetimini yalnızca teorik çözümleme akışı olarak değil, operasyonel kayıt yönetimi olarak da görmendir. Alan adı eklemek, İnternet'in dağıtık veritabanına yeni bilgi yazmak demektir.

Kısa Özet

DNS'e yeni bilgi eklemek, registrar üzerinden alan adı ve yetkili sunucu bilgilerini kaydettirmekle başlar. NS kayıtları ve glue record'lar, yeni alan adının İnternet genelinde çözümlenebilir hale gelmesini sağlar.

Uygulama Katmanı: Genel Bakış – P2P Uygulamalara Geçiş

Bu yol haritası slaytı, merkezi istemci-sunucu tabanlı servislerden daha dağıtık bir uygulama dünyasına geçtiğini gösterir. Burada önemli olan, P2P yaklaşımının yalnızca “sunucu yok” fikrine indirgenmemesidir. Asıl mesele, hizmet üretme kapasitesinin uç sistemlere dağıtılmasıdır. Bu nedenle P2P, yalnızca alternatif bir mimari değil, ölçeklenebilirlik sorununa verilen farklı bir mühendislik cevabıdır.

Önceki konularla bağ kurarsan şunu görürsün: istemci-sunucu modelinde yük merkezde toplanıyordu; P2P'de ise yük ve kapasite birlikte dağıtılır. Bu da özellikle büyük dosya dağıtımı ve geniş kullanıcı kitlesine hizmet verme senaryolarında çok kritik hale gelir.

Kısa Özet

Bu geçiş, uygulama mimarisinde merkezi yapıdan dağıtık yapıya yönelişi temsil eder. P2P'yi anlamak için asıl odak noktan, “yeni eşler sisteme yalnızca talep değil, aynı zamanda kapasite de getirir” fikri olmalıdır.

Eşler Arası (P2P) Mimari

P2P mimaride sürekli açık merkezi bir sunucu zorunlu değildir. Rastgele uç sistemler, yani peers, doğrudan birbirleriyle iletişim kurar. Her peer hem hizmet alabilir hem de hizmet verebilir. Slayttaki “self-scalability” vurgusu bu yüzden çok önemlidir: sisteme yeni katılan her eş, yalnızca yeni bir istemci değildir; aynı zamanda yeni bir yükleme kapasitesi kaynağıdır. Bu nedenle P2P yapılar, çok sayıda katılımcı olduğunda istemci-sunucu modeline göre daha avantajlı hale gelebilir.

Burada dikkat edilmesi gereken nokta, P2P'nin bedelsiz bir çözüm olmamasıdır. Slaytta da belirtildiği gibi yönetim daha karmaşıktır. Eşler aralıklı olarak bağlanabilir, IP adresleri değişebilir, güvenlik ve performans öngörülebilirliği zorlaşabilir. Yani bu mimari, merkezi kontrolün kolaylığını bırakıp dağıtık kapasitenin avantajını alır.

Bu kavramı şöyle düşünebilirsin: istemci-sunucu modelinde büyük bir depo tüm talepleri karşılar; P2P'de ise depolama ve dağıtım işi kullanıcıların kendilerine dağılır. Bu, sistemi ucuzlatabilir ve ölçekleyebilir; ama koordinasyonu zorlaştırır.

Kısa Özet

P2P mimari, hizmet üretme ve tüketme kapasitesini uç sistemlere dağıtır. Güçlü yanı kendi kendine ölçeklenebilmesidir; zor yanı ise güvenlik, yönetim ve kararlılığın daha karmaşık hale gelmesidir.

Dosya Dağıtımı: İstemci-Sunucu ve P2P

Bu slayt, bir dosyanın N sayıda kullanıcıya dağıtılmasında zamanın hangi kaynaklarla sınırlı olduğunu gösterir. Dosya boyutu F , sunucu upload kapasitesi u_s , peer upload kapasitesi u_i ve peer download kapasitesi d_i ile modellenir. Burada dikkat edilmesi gereken temel fikir şudur: dosya dağıtım süresi yalnızca dosya boyutuna değil, toplam yükleme ve indirme kapasitesine bağlıdır. Özellikle upload kapasitesi çoğu senaryoda kritik dar boğazdır.

İstemci-sunucu modelinde tüm yük büyük ölçüde sunucunun upload kapasitesine biner. Yani dosya ne kadar çok kişiye dağıtılacaksa, sunucunun her kullanıcı için veri göndermesi gerekir. Buna karşılık P2P'de dosyanın parçalarını alan eşler, aynı anda başka eşlere de yükleme yapmaya başlar. Böylece toplam sistem upload kapasitesi zamanla artar.

Bu slaytı anlamak için formülü ezberlemekten çok şu sezgiyi kurmalısın: dağıtım süresini belirleyen şey, “toplam kaç kişi var?” sorusundan çok, “bu kişilerin sistemi besleme kapasitesi var mı?” sorusudur.

Kısa Özet

Dosya dağıtım süresi, dosya boyutu kadar toplam upload ve download kapasitesine de bağlıdır. İstemci-sunucu modelinde yük merkezde kalırken, P2P modelinde eşler sisteme yeni dağıtım kapasitesi ekler.

Dosya Dağıtım Süresi: İstemci-Sunucu ve P2P Karşılaştırması

Bu karşılaştırmanın ana fikri çok nettir: istemci-sunucu mimarisinde dağıtım süresi, kullanıcı sayısı arttıkça büyür ve pratikte sunucu upload kapasitesi tarafından sınırlanır. Kitaptaki grafikte bu sürenin N arttıkça yaklaşık doğrusal biçimde yükseldiği gösterilir. Buna karşılık P2P’de, yeni kullanıcılar sadece tüketici değil aynı zamanda yeniden dağıtıcı olduğu için sistemin toplam kapasitesi de büyür. Bu nedenle P2P mimarisi daha iyi ölçeklenir. Kitaptaki şekil, belirli varsayımlar altında P2P dağıtım süresinin istemci-sunucuya göre her zaman daha düşük olduğunu gösterir.

Burada çok önemli olan, P2P’nin avantajının “sihirli bir optimizasyon” değil, yapısal bir özellik olmasıdır. Merkezi modelde sunucu tek başına taşır; dağıtık modelde ise taşınan yük çok sayıda eş tarafından paylaşılır. Bu yüzden P2P, katılımcı sayısı arttıkça daha anlamlı hale gelir.

Bu farkı şöyle yorumlayabilirsin: istemci-sunucu modelinde ek kullanıcılar yalnızca kuyruğu uzatır; P2P modelinde ek kullanıcılar kuyruğu uzatabilir ama aynı zamanda yeni servis noktaları da oluşturur.

Kısa Özet

İstemci-sunucu modelinde dağıtım süresi kullanıcı sayısı ile birlikte hızla büyür. P2P modeli ise katılımcıları aynı zamanda dağıtıcıya dönüştürdüğü için çok daha iyi ölçeklenebilir.

BitTorrent’e Giriş

BitTorrent, P2P dosya dağıtımının en bilinen örneklerinden biridir. BitTorrent dünyasında belirli bir dosyanın dağıtımına katılan tüm eşler topluluğuna torrent denir. Dosya eşit büyüklükte parçalara, yani chunks’a ayrılır. Bir peer ağa ilk katıldığında elinde hiç parça yoktur; zamanla farklı eşlerden parçalar indirir, aynı anda da indirdiği parçaları başkalarına yüklemeye başlar. Dosyanın tamamını alınca ister ağdan ayrılabilir, ister kalıp yüklemeye devam edebilir. Kitap bu davranışı ayrıntılı biçimde anlatır.

Burada anlamın gereken temel şey, BitTorrent’in “tek kaynaktan büyük dosya indirme” mantığını kırmasıdır. Sistem, dosyayı küçük parçalara bölerek farklı kaynaklardan eşzamanlı indirme ve eşzamanlı yükleme imkânı yaratır. Bu, hem verimi artırır hem de tek kaynağa bağımlılığı azaltır.

Bu yapının başarılı olabilmesi için yalnızca parçalara bölmek yetmez; hangi parçanın kimden isteneceği, kime veri verileceği ve bedavacıların (free riders) nasıl sınırlandırılacağı gibi kurallar da gerekir. Sonraki BitTorrent slaytları tam olarak bunu açıklar.

Kısa Özet

BitTorrent, dosyayı parçalara ayırıp bu parçaları çok sayıda eş arasında dağıtan bir P2P protokolüdür. Gücü, eşlerin aynı anda hem indirici hem de yükleyici rolü üstlenebilmesinden gelir.

BitTorrent: Dosya Parçaları ve Komşu Eşler

BitTorrent'te bir peer tüm ağla rastgele değil, genellikle kendi komşu eşleriyle (neighbors) veri alışverişi yapar. Elindeki parçaların listesi ve başkalarının sahip olduğu parçalar düzenli olarak paylaşılır. Böylece her peer, hangi komşusundan hangi parçayı isteyebileceğini bilir. Bu dağıtık koordinasyon çok önemlidir; çünkü merkezi bir denetleyici olmadan parça yayılımının dengeli ve verimli olması gerekir. Kitap, chunk temelli tasarımın BitTorrent'in başarısındaki ana mühendislik fikirlerinden biri olduğunu gösterir.

Burada dikkat edilmesi gereken nokta, dosyanın her peer tarafından aynı sırayla indirilmemesidir. Sistem parça çeşitliliğini korumaya çalışır. Eğer herkes aynı parçaları aynı anda isterse bazı parçalar çok yaygın, bazıları ise çok nadir olur ve dağıtım verimsizleşir.

Bu yüzden BitTorrent'te veri alışverişi yalnızca “kimde veri var?” sorusuyla değil, “hangi veri sistem genelinde nadir?” sorusuyla da şekillenir.

Kısa Özet

BitTorrent'te eşler dosyanın parçalarını komşularıyla dağıtık biçimde paylaşır. Asıl hedef yalnızca indirmek değil, parça çeşitliliğini koruyarak tüm torrent'in verimli çalışmasını sağlamaktır.

BitTorrent: Rarest-First Mantığı

Rarest-first, BitTorrent'in en kritik stratejilerinden biridir. Temel fikir şudur: bir peer, komşuları arasında en az sayıda bulunan parçaları öncelikli ister. Bunun amacı, sistem genelinde parçaların dağılımını dengeli tutmaktır. Slayttaki vurgu çok yerindedir: rarest-first olmasaydı herkes dosyanın baş tarafını indirir, bazı parçalar çok yaygın hale gelirken son parçalar darboğaz olurdu. Bu gerçeğe de dağıtık optimizasyon fikridir.

Burada dikkat etmen gereken şey, bu stratejinin bireysel değil sistemsal kazanç üretmesidir. Tek bir kullanıcı açısından herhangi bir parçayı almak benzer görünebilir. Ama ağ genelinde nadir parçaların korunması, dosyanın tamamının daha hızlı ve dengeli yayılmasını sağlar.

Bu yüzden BitTorrent'in zekâsı, yalnızca bir dosyayı bölmesinde değil; hangi parçanın ne zaman, hangi eşten isteneceğini akıllı seçmesinde yatar.

Kısa Özet

Rarest-first, sistemde en az bulunan parçaları önceliklendiren stratejidir. Amaç, parça dağılımını dengede tutmak ve bazı parçaların darboğaz haline gelmesini önlemektir.

BitTorrent: Tit-for-Tat ve Choking

BitTorrent yalnızca parçaları iyi dağıtmak zorunda değildir; aynı zamanda kullanıcıları katkı vermeye teşvik etmelidir. Tit-for-tat mantığı bu noktada devreye girer. Slaytta Alice'in kendisine en yüksek hızda veri gönderen en iyi dört peer'a upload yaptığı, diğerlerini ise choke ettiği anlatılıyor. Yani sistem, katkı verene öncelik verir. Bu davranış tamamen rastgele değildir; yaklaşık her 10 saniyede en iyi dört partner yeniden değerlendirilir. Böylece yükleme kapasitesi en verimli eşleşmelere yönlendirilir.

Burada “choke” kavramını iyi anlamalısın. Choke, belirli bir eşe veri göndermemek anlamına gelir. Bu bir ceza gibi görünse de aslında sınırlı upload kapasitesinin akıllıca paylaştırılmasıdır. Tit-for-tat, “sen bana veri veriyorsan ben de sana veririm” mantığıyla bedavacıları azaltmayı hedefler.

Kitap ayrıca bunun kusursuz ve tamamen sömürülemez bir mekanizma olmadığını, fakat BitTorrent'in başarısı için kritik olduğunu vurgular. Yani protokolün sosyal-ekonomik boyutu da teknik boyutu kadar önemlidir.

Kısa Özet

Tit-for-tat, katkı veren eşleri ödüllendirerek BitTorrent'te veri paylaşımını teşvik eder. Choking mekanizması sayesinde sınırlı upload kapasitesi, en faydalı eşleşmelere yönlendirilir.

BitTorrent: Optimistic Unchoke ve Dinamik Eşleşme

BitTorrent yalnızca mevcut en iyi dört partnere veri vererek çalışsaydı, yeni katılan eşlerin sisteme entegre olması zorlaşırdı. Bu yüzden optimistic unchoke mekanizması eklenmiştir. Slaytta her 30 saniyede bir rastgele bir peer seçilip ona da veri gönderildiği anlatılıyor. Amaç, yeni eşlere şans vermek, daha iyi bağlantı partnerleri keşfetmek ve sistemin donuk bir hâl almasını önlemektir.

Bu mekanizma çok zekicedir. Çünkü sistem tamamen performans geçmişine göre kilitlenmez; belirli aralıklarla keşif yapar. Eğer bu rastgele seçilen peer iyi bir ortak çıkarsa, zamanla karşılıklı en iyi partnerlerden biri olabilir. Slayttaki dinamik süreç tam olarak bunu gösteriyor: rastgele deneme, karşılıklı veri alışverişi, sonra iyi bir ticaret partnerine dönüşme.

Bunu ağ protokollerinde denge kurma örneği olarak görmelisin: yalnızca sömürüye karşı koruma yetmez; aynı zamanda yeni katılımcıların sisteme girmesi ve iyi bağlantıların keşfedilmesi gerekir.

Kısa Özet

Optimistic unchoke, yeni eşlerin sisteme girmesini ve daha iyi partnerlerin keşfedilmesini sağlar. BitTorrent böylece hem verimliliği korur hem de yeni katılanların dışlanmasını önler.

Uygulama Katmanı: Genel Bakış – Video Akışı ve CDN'e Geçiş

Bu geiş slaytı, P2P dosya dağıtımından video akışı ve ierik dağıtım ağılarına (CDN) getiğini gösterir. Burada dikkat etmen gereken şey, video akışının klasik dosya indirmeden farklı kalite gereksinimlerine sahip olmasıdır. Kullanıcı tüm dosyayı önce indirip sonra izlemek istemez; akış, oynatma ile veri gelişinin paralel ilerlemesini gerektirir. Bu da throughput, gecikme ve ierik dağıtımı konularını ok daha kritik hale getirir.

Önceki P2P bölümünden gelen önemli miras şudur: büyük ölekli ierik dağıtımı tek merkezle zorlaşır. Video akışı ve CDN konusu da aynı problemi başka bir mühendislik özümüyle ele alır; bu kez dağıtım eşlere değıl, coğrafi olarak yayılmış sunucu kümelerine verilir.

Kısa Özet

Video akışı ve CDN bölümü, büyük ölekli ierik dağıtımını başka bir açıdan ele alır. Burada temel mesele, ok sayıda kullanıcıya yüksek bit hızlı ieriğı düşük gecikme ve kabul edilebilir kaliteyle ulaştırmaktır.

Video Akışı ve CDN’ler: Bağlam

Bu slayt, video trafiğinin neden özel bir mühendislik problemi olduğunu açıklar. Sunuda, Netflix, YouTube ve Amazon Prime gibi hizmetlerin toplam İnternet trafiğinin büyük bölümünü oluşturduğu vurgulanıyor. Kitap da 2020 itibarıyla streaming video’nun İnternet trafiğinin yaklaşık yüzde 80’ine ulaştığını belirtir. Bunun iki ana zorluğu vardır: ölek ve heterojenlik. Ölek, ok sayıda kullanıcıya aynı anda hizmet verebilme problemidir. Heterojenlik ise kullanıcıların bant genişliği, gecikme, cihaz ve erişim türü bakımından ok farklı koşullara sahip olmasıdır.

Burada asıl mesele, videonun yüksek bit oranıdır. Kitap, sıkıştırılmış videonun 100 kbps gibi düşük değıerlerden 4 Mbps ve üstüne, hatta 4K için 10 Mbps’nin üzerine çıkabildiğini vurgular. Ayrıca en önemli performans ölçütünün ortalama uçtan uca throughput olduğunu açıka söyler. Yani video hizmetinin kalitesi büyük ölüde verinin istemciye yeterli hızda ulaşp ulaşmamasına bağlıdır.

Bu nedenle video akışı problemi, yalnızca “bir dosya gönder” problemi değıldir; ierik kodlama, ağ koşulları ve dağıtım altyapısı birlikte düşünölmelidir.

Kısa Özet

Video akışı, yüksek trafik üretmesi ve kullanıcı koşullarının ok değışken olması nedeniyle özel bir uygulama katmanı problemidir. En kritik ağ ölçütü, videoyu akıcı oynatacak kadar yüksek ve kararlı throughput sağlayabilmektir.

İnternet Videosu ve Performans Ölütleri

İnternet videosunda temel ortam önceden kaydedilmiş videodur. Kullanıcı videoyu ister, sunucu ieriğı akış hâlinde gönderir ve istemci tarafı bunu oynatır. Burada dikkat edilmesi gereken nokta, video deneyimini belirleyen tek şey toplam dosya boyutu değıldir. Sıkıştırma oranı, bit hızı, oynatma tamponu ve ağıın o andaki verim kapasitesi birlikte sonucu belirler.

Kitap özellikle bit oranı ile görüntü kalitesi arasında doğrudan bir değiş-tokuş olduğunu vurgular: bit rate yükseldikçe kalite de artar, ama ağdan daha fazla kapasite istenir.

Bu slayt bağlamında şu sezgiyi kurmalısın: video akışında sorun çoğu zaman “paketlerden bazıları kayboldu” değil, “veri yeterince hızlı gelmediği için oynatma tamponu boşaldı” problemidir. Yani throughput düşerse video donar, kalite azalır veya oynatma takılır. Bu yüzden stored video streaming’de throughput, gecikmeden bile daha baskın bir kalite ölçütü olabilir.

Buradan DASH ve uyarlanabilir akış (adaptive streaming) fikrine zemin hazırlanır. Çünkü farklı kullanıcılar için aynı bitrate her zaman uygun değildir.

Kısa Özet

İnternet videosunda kalite, büyük ölçüde bit rate ile ilişkilidir; ama bu bit rate’in ağ tarafından sürdürülebilmesi gerekir. Bu yüzden stored video streaming’de en kritik ağ ölçütlerinden biri uçtan uca throughput’tur.

HTTP Akışı ve DASH

DASH (Dynamic Adaptive Streaming over HTTP), video akışında uyarlanabilir kaliteyi mümkün kılan çok önemli bir fikirdir. Temel mantık şudur: video farklı kalite düzeylerinde, küçük parçalar hâlinde hazırlanır. İstemci, o anki ağ koşullarına göre hangi kalite seviyesindeki bir sonraki parçayı isteyeceğine karar verir. Böylece sabit tek bir bitrate yerine, koşula göre artan veya azalan bir akış elde edilir. Bu yaklaşım, heterojen kullanıcı ve değişken ağ koşullarına çok daha iyi uyum sağlar. Bu konu sunuda video akışı/CDN bağlamında yer alırken, kitap da modern video dağıtımının uygulama düzeyinde akıllı parça seçimleriyle çalıştığını açıklar.

Burada dikkat edilmesi gereken nokta, kontrolün büyük ölçüde istemci tarafında olmasıdır. İstemci ölçüm yapar, throughput tahmini üretir ve buna göre bir sonraki parçanın kalitesini seçer. Bu nedenle DASH, yalnızca video dosyasını bölmek değil; istemciye ağ koşullarına göre karar verme sorumluluğu vermektir.

Bu kavramı şöyle düşünebilirsin: tek beden kıyafet herkese uymuyorsa, sistem farklı bedenler üretir ve kullanıcıya uygun olanı seçer. DASH de videoyu tek sabit kalite yerine çoklu kalite seçenekleriyle sunar.

Kısa Özet

DASH, videoyu küçük parçalar ve çoklu kalite sürümleri hâlinde sunarak istemcinin ağ koşullarına göre uyarlanabilir kalite seçmesini sağlar. Böylece video akışı daha kararlı ve kullanıcı koşullarına daha uygun hâle gelir.

Neden CDN?

CDN’in temel motivasyonu, kullanıcı ile içerik arasındaki fiziksel ve topolojik mesafeyi azaltmaktır. Eğer tüm video tek bir dev veri merkezinden sunulsaydı, uzak kullanıcılar için

gecikme artar, dar boğaz bağlantılar büyür ve altyapı çok çabuk yük altında kalırdı. CDN, içeriği coğrafi olarak dağıtılmış sunucu kümelerine yerleştirerek bu problemi çözer. Bu sunucular bazı yönlerden büyük ölçekli önbellekler gibi çalışır. Kitap da bu benzetmeyi özellikle yapar.

Burada iki temel yaklaşım öne çıkar: enter-deep, yani sunucuları erişim ISP’lerinin derinliklerine kadar yerleştirmek; veya bring-home, yani daha az sayıda ama büyük stratejik küme kurmak. Her iki yaklaşımın da maliyet, yönetim ve performans açısından farklı sonuçları vardır. Ama ortak amaç aynıdır: içeriği kullanıcıya yaklaştırmak.

Bu yüzden CDN’i sadece “çok sayıda sunucu” diye düşünmemelisin. Asıl mesele, içeriğin ağ üzerinde akıllıca konumlandırılmasıdır.

Kısa Özet

CDN, içeriği kullanıcıya yakın sunucu kümelerine dağıtarak performansı ve ölçeklenebilirliği artırır. Temel fikir, içeriği merkezde tutmak yerine ağı farklı bölgelerine yaymaktır.

CDN Çalışması ve DNS ile Yönlendirme

Bir CDN’in en ilginç taraflarından biri, kullanıcıyı fark ettirmeden uygun sunucuya yönlendirebilmesidir. Kitap burada özellikle DNS tabanlı yönlendirmeyi açıklar. Kullanıcının tarayıcısı bir video URL’si istediğinde CDN, isteği “yakalar”, istemci için uygun bir CDN kümesini belirler ve istemciyi o kümeye yönlendirir. Bu seçim coğrafi yakınlık, ağ durumu, yük dağılımı veya başka politikalarla yapılabilir. DNS, bu yönlendirme işinde merkezi rol oynar.

Burada dikkat edilmesi gereken nokta, DNS’in yalnızca isim çözümleyen bir servis değil, aynı zamanda trafik yönlendirme mekanizmasının parçası haline gelebilmesidir. Kullanıcı aynı alan adını girse bile, farklı kullanıcılar farklı CDN sunucularına yönlendirilebilir. Bu, uygulama katmanı ile dağıtım altyapısının ne kadar iç içe geçtiğini gösterir.

Bu yapıyı anlamak için şunu düşün: kullanıcı videoyu tek bir “site”den aldığını zanneder, ama aslında DNS ve CDN mekanizmaları onu en uygun içerik kümesine yönlendiriyor olabilir.

Kısa Özet

CDN’ler, DNS yardımıyla kullanıcıyı uygun içerik sunucusuna yönlendirir. Böylece aynı hizmet adı altında farklı kullanıcılar, ağ açısından en uygun sunucu kümelerinden içerik alabilir.

Socket Programlamaya Geçiş

Bu geçişte artık teoriden uygulamaya, yani ağ programlamasının kod tarafına geliyorsun. Şimdiye kadar süreç, socket, istemci-sunucu, taşıma protokolleri ve uygulama katmanı mesaj biçimleri kavramsal olarak işlendi. Socket programming ile bunların nasıl gerçek koda dönüştüğünü görmeye başlıyorsun. Bu çok önemli bir aşamadır; çünkü soyut görünen birçok kavram burada somut API çağrılarına dönüşür.

Burada zihninde şu ayrımı korumalısın: programlama örnekleri protokol teorisinin yerine geçmez; onu görünür hale getirir. `socket()`, `connect()`, `bind()`, `listen()`, `accept()`, `send()`, `recv()` gibi çağrılar, daha önce konuştuğun süreçler arası iletişimin gerçek uygulama yüzüdür.

Kısa Özet

Socket programlama, uygulama katmanı kavramlarının kod düzeyindeki karşılığıdır. Burada amaç yalnızca çalışan örnek görmek değil, teoride öğrendiğin yapıların API çağrılarına nasıl dönüştüğünü anlamaktır.

UDP ile Socket Programlama

UDP ile programlama, bağlantısız iletişimin ne anlama geldiğini çok net gösterir. İstemci tarafında bir UDP socket oluşturulur, hedef sunucunun IP adresi ve port numarası belirtilerek datagram gönderilir. Sunucu tarafında ise belirli bir porta bind edilen socket gelen datagramları dinler. Burada dikkat edilmesi gereken temel fark, TCP'deki gibi önceden bağlantı kurulmamış olmasıdır. Bu yüzden her gönderimde hedef adres bilgisi önemlidir; alıcı tarafta da gelen datagramın hangi istemciden geldiği bilgisi ayrı olarak alınır. Kitap UDP socket programlamasını özellikle bu bağlantısızlık üzerinden anlatır.

Bu modelin sadeliği avantajdır; ama uygulamaya daha fazla sorumluluk bırakır. Paket kaybolursa, sırası bozulursa veya hiç ulaşmazsa UDP bunu uygulama adına çözmez. Bu yüzden UDP socket programlama, ağın her zaman kusursuz davranmadığını varsayarak düşünülmelidir.

Burada önemli sezgi şudur: UDP'de “mesaj” hissi daha belirgindir; her datagram ayrı bir birim gibi ele alınır. TCP'de ise daha çok sürekli bir bayt akışı düşüncesi vardır.

Kısa Özet

UDP socket programlama, bağlantısız ve hafif iletişim modelini görünür kılar. Basit ve hızlıdır; ancak güvenilirlik ve sıra garantisi uygulamaya bırakılmıştır.

TCP ile Socket Programlama

TCP socket programlama, bağlantı yönlimli iletişimin uygulama düzeyindeki karşılığıdır. İstemci `socket()` ile bir TCP socket oluşturur ve `connect()` çağrısıyla sunucunun IP/port bilgisine bağlanır. Sunucu ise önce bir welcoming socket oluşturur, `bind()` ile porta bağlanır, `listen()` ile dinlemeye geçer ve `accept()` ile gelen istemci bağlantılarını kabul eder. Kitap, öğrencilerin özellikle welcoming socket ile connection socket'i karıştırmaması gerektiğini vurgular. Çünkü bunlar farklı roller taşır.

Burada çok önemli olan şey, three-way handshake'in uygulama tarafından doğrudan kodlanmamasıdır. `connect()` ve `accept()` çağrılarının arkasında, taşıma katmanı düzeyinde TCP bağlantı kurulumu görünmez biçimde gerçekleşir. Uygulama bunu doğrudan yönetmez; ancak sonuçlarından yararlanır. Sonra istemci ve sunucu, güvenilir bir bayt akışı üzerinden veri gönderip alır.

Bu programlama modelini anlamak, TCP'nin neden “stream-oriented” dendiğini de gösterir. Uygulama, çoğu zaman adres taşımak zorunda kalmadan açık bağlantı üzerinden karşılıklı veri alışverişi yapar.

Kısa Özet

TCP socket programlama, bağlantı kurulumu ve güvenilir bayt akışı mantığını koda dönüştürür. Welcoming socket ve bağlantıya özel socket ayrımı, TCP sunucu mantığını anlamanın anahtarıdır.

Örnek Uygulama: TCP İstemcisi

Sunudaki Python TCP istemcisi örneği, TCP tarafındaki istemci davranışını çok sade biçimde gösterir. İstemci önce sunucu adını ve port numarasını bilir, sonra `socket (AF_INET, SOCK_STREAM)` ile TCP socket'i oluşturur. `connect ()` çağrısı ile bağlantıyı başlatır. Sonra kullanıcıdan aldığı metni `send ()` ile gönderir, `recv ()` ile sunucunun cevabını alır ve bağlantıyı kapatır. Bu örnekte kritik fark, UDP'den farklı olarak istemcinin her veri gönderiminde hedef adres belirtmek zorunda olmamasıdır; çünkü bağlantı kurulunca bu bağlam zaten socket üzerinde tutulur.

Bu örneği incelerken şu bağlantıyı kuralırsınız: daha önce konuştuğunuz istemci-sunucu modeli artık kod halinde görünür oldu. `connect ()` istemcinin iletişimi başlatan taraf olduğunu, `recv ()` ise istemcinin sunucudan dönen veriyi beklediğini gösterir.

Ayrıca bu kod, TCP'nin uygulamaya sunduğu rahatlığı da görünür kılar. Adresleme, sıra, yeniden iletim gibi konular istemci kodunda açıkça görünmez; çünkü TCP bunları taşıma katmanında yönetir.

Kısa Özet

TCP istemci örneği, bağlantıyı başlatma, veri gönderme ve yanıt alma akışını gösterir. TCP'nin sunduğu güvenilirlik ve bağlantı bağlamı sayesinde istemci kodu görece sade görünür.

Örnek Uygulama: TCP Sunucusu

TCP sunucu örneği, istemciye göre biraz daha zengindir çünkü iki düzeyde socket yönetir. Önce `serverSocket` adı verilen welcoming socket oluşturulur, belirli bir porta bağlanılır ve dinlemeye geçilir. `accept ()` çağrısı geldiğinde, o istemciye özel yeni bir `connectionSocket` oluşur. Veri bu yeni socket üzerinden okunur ve cevap yine bunun üzerinden gönderilir. İstemciyle iş bitince `connectionSocket` kapanır; ama welcoming socket açık kalır ve yeni istemcileri kabul etmeye devam eder. Kitap bu ayrımı özellikle vurgular.

Burada dikkat edilmesi gereken nokta, sunucunun sürekli açık ve sürekli dinlemede olmasıdır. Bu, istemci-sunucu mimarisinde “always-on server” fikrinin programlama düzeyindeki karşılığıdır. Sunucu döngü içinde yeni istekleri bekler ve her gelen bağlantı için ayrı bir etkileşim yürütür.

Bu örnek ayrıca TCP'nin iki yönlü iletişim mantığını da gösterir. Sunucu yalnızca veri almaz; aldığı veriyi işleyip geri gönderir. Yani bağlantı tek yönlü boru değil, çift yönlü bir akıştır.

Kısa Özet

TCP sunucu örneği, dinleyen socket ile bağlantıya özel socket ayrımını gösterir. Sunucu sürekli dinler, her istemci için ayrı bir bağlantı socket'i üretir ve veri alışverişini bu socket üzerinden yürütür.

Bölüm 2 Özeti

Bu özet slaytı, tüm bölümün omurgasını çok güzel toplar. Uygulama mimarileri olarak istemci-sunucu ve P2P'yi gördün. Uygulama servis gereksinimleri olarak güvenilirlik, bant genişliği ve gecikmeyi değerlendirdin. İnternet taşıma servis modelinde TCP'nin bağlantı yönelimli ve güvenilir, UDP'nin ise bağlantısız ve güvenilir olmayan yapı sunduğunu gördün. Ayrıca HTTP, SMTP/IMAP, DNS, BitTorrent, video akışı, CDN ve socket programlama gibi somut örneklerle bu ilkeleri pekiştirdin.

Burada asıl büyük resim şudur: uygulama katmanı, yalnızca “kullanıcıya yakın” olduğu için değil, tüm ağ mimarisinin neden var olduğunu gösterdiği için önemlidir. Ağın diğer katmanları, sonunda burada çalışan uygulamalara hizmet eder.

Kısa Özet

Bu bölüm, ağ uygulamalarını hem kavramsal hem de pratik açıdan ele aldı. En önemli kazanç, bir uygulamanın mimari seçimi, protokol davranışı ve taşıma katmanı gereksinimi arasındaki ilişkiyi görebilme.

Bölüm 2 Özeti – Protokoller Hakkında Öğrendiğin En Önemli Şey

Son özet slaytındaki en değerli vurgu, “aslında protokolleri öğrendin” cümlesidir. Çünkü tüm bölüm boyunca işlenen HTTP, SMTP, DNS, BitTorrent ve socket örnekleri farklı görünse de ortak bir çekirdeği paylaşır: tipik request/reply veya mesaj alışverişi, yapılandırılmış message format'ları, payload + header ayrımı ve belirli davranış kuralları. Ayrıca merkezi/dağıtık yapı, stateless/stateful yaklaşım, ölçeklenebilirlik ve güvenilir/güvenilir olmayan aktarım gibi temalar tekrar tekrar karşına çıktı.

Bu slaytı bölümün felsefi kapanışı gibi okuyabilirsin. Artık bir uygulama katmanı protokolüne baktığında yalnızca komutları değil, onun altında yatan mimari tercihleri, servis beklentilerini ve ölçeklenme stratejilerini de görebilmelisin.

Kısa Özet

Bu bölümün en büyük kazanımı, tek tek protokollerden çok protokol düşünme biçimini öğrenmektir. Mesaj yapısı, durum yönetimi, mimari tercih ve servis gereksinimi birlikte düşünüldüğünde ağ uygulamaları gerçek anlamda anlaşılır.

Ek Slaytlar – Socket’te Timeout Mantığı

Ek slaytlardaki timeout vurgusu özellikle programlama açısından çok değerlidir. Ağ uygulamalarında program bazen yalnızca veri gelmesini beklemeyebilir; birden fazla olayı aynı anda beklemek zorunda kalır. Örneğin ya karşı taraftan yanıt gelmeli ya da belirli süre dolmalıdır. Burada bu durumun `settimeout()` ve Python’daki exception mantığıyla gösterilmesi çok yerindedir. Timeout’lar ağ programlamasında çok yaygın kullanılır; özellikle yanıt gelmemesi ile uzun süre bloklanma problemini ayırt etmek için gereklidir.

Burada önemli olan, timeout’un ağda gerçekten hata olduğu anlamına gelmek zorunda olmamasıdır. Bazen paket kaybolmuştur, bazen gecikme beklenenden uzundur, bazen de hedef hiç cevap vermiyordur. Uygulama açısından önemli olan şey, sonsuza kadar beklememek ve belirli bir süre sonra kontrolü geri almaktır.

Bu kavram, sonraki taşıma katmanını ve güvenilir veri aktarımını konularına da köprü kurar. Çünkü timer kullanımı, ağ protokollerinin birçok yerinde temel araçtır.

Kısa Özet

Timeout, uygulamanın sonsuza kadar beklemesini engelleyen kritik bir ağ programlama aracıdır. Yanıt gelmemesi durumunda programın kontrollü biçimde tepki verebilmesini sağlar.

Python `try-except` ve Socket Timeout

Timeout ile birlikte exception handling mantığını anlamak gerekir. Python’da bir `try-except` bloğu, belirli bir kod parçası çalışırken oluşabilecek istisnaları yakalayıp uygun tepki vermeyi sağlar. Socket programlamada bu özellikle önemlidir; çünkü `recv()` gibi çağrılar zaman aşımına uğrayabilir ve uygulama bu durumu açıkça ele almak zorundadır. Sunudaki örnek, `settimeout()` çağrısının belirli socket işlemlerine zaman sınırı getirdiğini ve süre dolarsa exception oluştuğunu gösteriyor.

Burada dikkat etmen gereken nokta, exception yakalamanın “hata saklamak” değil, beklenen olağan dışı durumları kontrollü biçimde yönetmek olduğudur. Ağlar kusursuz değildir; o yüzden programların da kusursuz ağ varsayımıyla yazılması doğru olmaz.

Bu bakış açısı seni daha dayanıklı ağ uygulamaları yazmaya götürür. Çünkü gerçek sistemlerde bekleme, gecikme, cevap vermeme ve bağlantı sorunları olağandır.

Kısa Özet

`try-except` yapısı, timeout gibi ağ kaynaklı istisnaları kontrollü biçimde ele almayı sağlar. Sağlam ağ uygulamaları yazmak için yalnızca normal akışı değil, anormal durumları da kodlaman gerekir.
